

Spielerverfolgung durch große Agentengruppen in komplexen 3D Szenen

Verfasser:	Henry Mark Hinsberger
Aufgabensteller/Prüfer:	Prof. Dr. Christoph Bichlmeier
Arbeit vorgelegt am:	25.10.2026
Fakultät:	Fakultät für Informatik
Studiengang:	Informatik-Game Engineering
Matrikelnummer:	435787
Betreuer	Prof. Dr. Christoph Bichlmeier

Kurzbeschreibung

Folgende Arbeit befasst sich mit der Konzipierung, sowie der Implementierung eines Algorithmus zur Spielerverfolgung durch große Agentengruppen. Ziel ist es, dass im Rahmen eines Videospiels ein von dem Spieler gesteuerter Character von hunderten bis tausenden computergesteuerten Gegnern verfolgt wird. Stattfinden soll dieses Szenario in einer komplexen 3D-Szene. Dabei soll beschrieben werden, welche Algorithmischen Prinzipen angewandt werden können, um diese Problemstellung performant zu lösen und ein immersives Spieleerlebnis zu ermöglichen. Die Implementierung wird nachvollziehbar dargelegt, auf Probleme und mögliche Lösungen wird eingegangen. Die Arbeit befasst sich hauptsächlich mit der technischen Umsetzung, geht aber auch auf Game Design Aspekte ein, insofern diese für die Umsetzung relevant sind.

VORWORT:

Ich möchte mich herzlich bei Prof. Dr. Christoph Bichlmeier für die Betreuung und den Vorschlag für das Thema bedanken. Mein Dank gilt außerdem meiner Familie, die mich immer unterstützt hat.

INHALTSVERZEICHNIS

1 EINLEITUNG 5

1.1 DEFINITIONEN 6

1.2 LITERATUR 9

1.3 TECHNISCHE DETAILS 10

2 VEKTORFELD NAVIGATION 11

3 ERSTELLEN DES GRAPHEN 12

3.1 INTERSECTION TEST 16

3.2 IMPLEMENTIERUNG 19

4 PATHFINDING ALGORITHMUS 21

4.1 A* ALGORITHMUS 21

4.2 VEKTOREN BERECHNEN 27

5 PERFORMANCE 28

5.1 ERSTELLEN DES GRAPHEN 28

5.2 A* 30

6 OPTIMIERUNGEN 34

6.1 PORTALE 34

6.2 DYNAMISCHE ZELLGRÖßE 37

6.3 DYNAMISCHE TICKS 38

6.4 NAHBEREICH LINE OF SIGHT NAVIGATION 39

6.5 MERGING 40

7 GAME DESIGN 43

7.1 ALTERNATIVE PFADE 43

7.2 VERHALTEN DER AGENTEN 48

7.3 ANIMATION UND MODELLE 48

8 ZUSAMMENFASSUNG 50

9 ABBILDUNGSVERZEICHNIS 51

10 LITERATURVERZEICHNIS 52

11 ERKLÄRUNGEN 53

11.1 SELBSTSTÄNDIGKEITSERKLÄRUNG 53

11.2 ERMÄCHTIGUNG 53

1 EINLEITUNG

Von einer Horde an Gegnern verfolgt zu werden, ist ein sehr beliebtes und immer wiederkehrendes Thema in Videospielen. Von Call of Duty bis Days Gone. Ganz auf sich gestellt vor den übermächtigen bösen Horden getrieben durch eine fantastische Welt zu laufen, ist ein mächtiger Adrenalinstoß.

Um diese Vorstellung auf den Bildschirmen der Spieler Wirklichkeit werden zu lassen, haben Entwickler sich einiges einfallen lassen müssen. Schließlich kann ein Spiel, dass für eine einstellige Zahl an Gegnern ausgelegt ist, unmöglich hundert, zweihundert oder gar tausend Gegner bewältigen. Und das Ganze noch in einer komplexen Welt, mit dynamischen Elementen, in der dabei ein möglichst ungestörtes immersives Spielerlebnis aufkommen soll.

Diese Arbeit befasst sich mit einem allgemeinen Szenario, in dem ein von einem Spieler gesteuerter Character von einer großen Zahl an computergesteuerten Agenten verfolgt wird. Diese haben es als Ziel, die Position des Spielers zu erreichen, während dieser sich weiter bewegen kann. Dabei soll es möglich sein, dass sich eine große Zahl an Agenten gleichzeitig in der Szene befindet. Es soll hier keine feste Grenze der zu erreichenden Agenten festgelegt werden. Ziel ist vielmehr einen Algorithmus zu entwerfen, der eine beliebig große Anzahl bewältigen kann, ohne die Performance des Programms zu beeinträchtigen.

Des Weiteren soll diese Verfolgungsjagd in einer komplexen dreidimensionalen Szene erfolgen. Der Algorithmus soll dabei in der Lage sein, unter verschiedenen geometrischen Gegebenheiten zu funktionieren.

Die Kapitel der Arbeit sind nach folgendem Schema aufgebaut:

- Zu Beginn werden die in einem Bereich bestehenden Problemstellungen erläutert. Spezifisch wird darauf eingegangen welche Schwierigkeiten sich aus der Fragestellung dieser Arbeit ergeben.
- Darauffolgend werden dann mögliche Lösungsansätze, sowie deren Vor- und Nachteile erläutert.
- Zuletzt wird dann ein Ansatz, sowie dessen Implementierung detailliert besprochen. Der so beschriebene Ansatz ist auch gleichzeitig immer die Lösung, die im Rahmen dieser Arbeit umgesetzt wurde.

Manche der Themen in Kapitel 7 „Game Design“ wurden im Rahmen dieser Arbeit nicht umgesetzt, sondern sind lediglich Gedanken und Ausführungen, die sich bei der Recherche oder am Rande der sonstigen Arbeiten ergeben haben. In diesem Fall ist der präsentierte Ansatz der von mir als am vielversprechendsten ausgewählte.

1.1 DEFINITIONEN

Die folgenden Abschnitte beinhalten Definitionen, die für diese Arbeit relevant sind. Einige davon basieren auf, in der Spieleentwicklung gängigen Begriffen (z.B. Agenten), andere sind spezifisch im Kontext dieser Arbeit relevant (z.B. Spielerverfolgung).

1.1.1 SPIELER

Wird in dieser Arbeit von dem Spieler geredet, ist nicht der Mensch, der vor dem Eingabegerät sitzt, gemeint, sondern der Character in der Szene, der von diesen Inputs gesteuert wird. Dabei wird der Spieler durch kontinuierliche Inputs durch die Szene bewegt, mit dem Ziel nicht von den Agenten erwischt zu werden.

Der Spieler besitzt die folgenden für diese Arbeit relevanten Eigenschaften:

- Die Position des Spielers pos_s im Welt-Koordinatensystem, repräsentiert durch einen Vektor:

$$pos_s = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- Die Zelle, in der sich der Spieler befindet (siehe hierzu Kapitel 2 Vektorfeld Navigation), repräsentiert durch die Indizes dieser Zelle:

$$cell_s = \begin{pmatrix} i_x \\ i_y \\ i_z \end{pmatrix}$$

Etwaige Fähigkeiten, die ein Spieler besitzt, vor allem jene, die seine Position auf eine nicht vorhersehbare Weise ändern können, werden in dieser Arbeit nicht behandelt. Alle beschriebenen Algorithmen müssen aber mit plötzlichen Positionsänderungen umgehen können, seien sie durch den Spieler oder sonstiges Spielgeschehen ausgelöst.

1.1.2 AGENTEN

Agent ist ein in der Spieleentwicklung häufig verwendeter Begriff, der computergesteuerte nicht spielbare Charaktere (sog. NPCs) in einer Szene beschreibt. Gesteuert werden diese meistens durch einen Zustandsgraphen, deren Kanten (sprich „Zustandsübergänge“) an Bedingungen geknüpft sind, bspw. „Agent sieht Spieler„. Die Knoten repräsentieren den jeweiligen Zustand, bspw. „läuft auf Spieler zu“. Zu den Eigenschaften eines Agenten zählen folgende Dinge:

- Model (Bestimmt, wie der Agent im Spiel erscheint)
- Animationen (Laufanimation, Angriffsanimation, etc.)
- Verhalten (wie verhält sich der Agent gegenüber dem Spieler, anderen NPCs, etc.)
- Voicelines

Im Rahmen dieser Arbeit liegt der Fokus hauptsächlich auf dem Verhalten des Agenten mit Bezug auf den Spieler. Animationen und Models finden auch Erwähnung in Kapitel 7.3

„Animation und Modelle“. Das Hauptaugenmerk liegt auf der Spielsituation, in der der Agent den Spieler verfolgt. Für die Verfolgung sind folgende dem Agenten zugewiesene Eigenschaften relevant:

- Die Position des Agenten pos_a im Welt-Koordinatensystem, repräsentiert durch einen Vektor:

$$pos_a = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- Die Zelle, in der sich der Agent befindet (siehe hierzu Kapitel 2 „Vektorfeld Navigation"), repräsentiert durch die Indizes dieser Zelle:

$$cell_a = \begin{pmatrix} i_x \\ i_y \\ i_z \end{pmatrix}$$

  Diese leitet sich aus der Position des Spielers ab.
- Der Status des Agenten. In diesem Fall unterscheiden wir zwei relevante Zustände:
  - Idle: Der Agent verfolgt den Spieler bereits
  - Chasing: Der Agent hat den Spieler noch nicht entdeckt
- Die Distanz zum Spieler auf dem Optimalen Pfad zwischen Agent und Spieler. Dieser bestimmt unter Umständen das Verhalten und welche Methode der Zielführung dieser wählt.

Die Agenten sind in einer großen Stückzahl in der Szene vorhanden. Diese sind alle mit den gleichen Algorithmen zur Entscheidungsfindung und vor allem zum Pathfinding ausgestattet. Unterschiede in anderen Aspekten, wie beispielsweise ihren Models, sind für die Problemstellung nicht relevant.

1.1.3 SPIELERVERFOLGUNG

Die Spielerverfolgung ist das zentrale Thema dieser Arbeit. Dieser Begriff bedarf noch einiger Erläuterung.

Der Spieler steuert einen Character durch eine gegebene Szene. In welcher Form die Inputs erfolgen, welches Modell der Character hat, mit welchen Fähigkeiten dieser ausgestattet ist usw. spielt keine Rolle. Für diese Arbeit kann der Spieler bzw. sein Character auf eine Größe reduziert werden, nämlich seine Position. Diese ändert sich kontinuierlich.

Die Agenten haben das Ziel, den Spieler zu erreichen, also sich physisch direkt neben dem Spieler zu befinden. Diese Arbeit betrachtet ausdrücklich keine Fälle, in denen die Agenten den Spieler mit Distanzwaffen angreifen und nur bis zu einer gewissen Distanz zum Spieler navigieren müssen. Auch die Geschwindigkeit des Spielers und der Agenten zueinander ist nicht ausschlaggebend.

Die Agenten haben, wenn sie den Spieler verfolgen zu jeder Zeit exaktes Wissen über dessen Position. Sie benötigen dafür keine Sichtlinie oder sonstige Methoden, um an Informationen zu gelangen.

Es ist außerdem Voraussetzung, dass alle sich in der Szene befindlichen Agenten gleichzeitig zum Spieler navigieren können. Wenn der Spieler stehen bleibt, sollten also nach einer bestimmten Zeit alle Agenten sich in seiner unmittelbaren Umgebung sammeln. Falls nicht anders angegeben, sollen die Agenten dabei den Optimalen Pfad nehmen. Die Kollisionsvermeidung zwischen den Agenten wird hier nicht behandelt.

1.1.4 SZENE

Ist in dieser Arbeit von der Szene die Rede, ist damit nicht der Begriff gemeint, wie er oft im Umfeld von Game Engines verwendet wird, also die Gesamtheit aller zu einem Zeitpunkt geladenen Objekte. Gemeint ist die Geometrie der Szene minus aller Akteure (Spieler und Agenten). Dabei ist was die Szene repräsentiert (Stadt, Wald, etc.) nicht von Bedeutung. Relevant ist die Szene, weil sie die Oberfläche bildet, auf der sich alle Akteure bewegen.

Alle in dieser Arbeit beschriebenen Algorithmen sind für komplexe dreidimensionale Szenen konzipiert. In der Arbeit wird mit einer Testszene gearbeitet, die sowohl ein Haus als auch eine freie Fläche hat. Der Anspruch ist aber, dass die Szene ausgetauscht werden kann, und mit wenig bis gar keinem Aufwand alles weiter funktioniert.

Der Aufwand bei der Erstellung, unabhängig von der Modellierung selber, beschränkt sich auf das manuelle Platzieren der Portale (siehe Kapitel 6.1 „Portale“). Die restlichen Vorbereitungen sind algorithmisch umgesetzt. Für ein optimales Ergebnis sollte jede Geometrie, die nicht betreten werden kann, keinen Boden haben, da sonst unnötige Zellen angelegt werden.

1.1.5 GAMEPLAY

Diese Arbeit befasst sich nicht ausführlich mit den Game Design Aspekten dieser Anwendung. Einige Ausführungen zu diesem Thema finden sich dennoch in Kapitel 7 „Game Design“. Die beschriebenen Algorithmen können auf verschiedenen Gameplay-Szenarien angewandt werden. Die in dieser Arbeit beschriebenen Ansätze wurden trotzdem im Kontext einer konkreten potenziellen Anwendung erarbeitet. Falls nicht anders angegeben, ist dieses potenzielle Spiel eine Zombiespiel, ähnlich „Days Gone“. Nicht nur hat dieses Genre als eine seiner wichtigsten Mechaniken die Zombies (Agenten), die den Spieler unermüdlich jagen. Diese haben außerdem keine Waffen und müssen den Spieler im Nahkampf angreifen. Zu guter Letzt sind diese Zombies häufig in großen Horden unterwegs, was genau das Szenario ist, mit dem sich diese Arbeit befasst.

Alle Aspekte werden trotzdem von diesem Inhalt unabhängig beschrieben. Die Agenten können mit verschiedensten Modellen und Animationen ausgestattet werden und die beschriebenen Algorithmen funktionieren auf die gleiche Art und Weise. Egal ob third-Person oder first-

Person, die Agenten sollen ihren Weg zum Spieler finden, egal ob durch einen Wald oder eine Stadt.

1.2 LITERATUR

Die primäre Quelle und Inspiration für diese Arbeit ist der 2001 von Elijah Emerson in dem Buch „Game AI Pro“ veröffentlichte Artikel: „Crowd Pathfinding and Steering Using Flow Field Tiles“. Dieser beschreibt einen ähnlichen Ansatz, wie jenen, der in dieser Arbeit verfolgt wird, mit dem entscheidenden Unterschied, dass Emerson die Anwendung für ein zwei-dimensionales Spielfeld beschreibt und sich diese Arbeit mir komplexen 3D Szenen beschäftigt [1].

Die „Game AI Pro“ Reihe ist im Allgemeinen eine sehr ergiebige Quelle. Auch das Kapitel „Pathfinding Architecture Optimizations“ von Steve Rabin und Nathan R. Sturtevant hat einige sehr relevante Hinweise für effizientes Pathfinding, von denen auch einige in dieser Arbeit Beachtung gefunden haben [2].

Andere Artikel wie der von Jordan T. Thayer und Wheeler Ruml: „Faster Than Weighted A*: An Optimistic Approach to Bounded Suboptimal Search“, indem die Autoren Optimierungsansätze beschreiben, die von dem Optimalen A* Ergebnis abweichen, sind vielversprechend, haben aber im Umfang dieser Arbeit keine Beachtung gefunden [3].

Für den in dieser Arbeit verfolgten Ansatz war auch das Paper von Thomas Akenine-Möller „Fast 3D Triangle-Box Overlap Testing“ von Bedeutung. Die in Kapitel 3 beschrieben Algorithmen basieren weitestgehend auf seinem Verfahren [4].

Leider dokumentieren nicht alle Entwickler ihr Wissen in kondensierter Form als Artikel oder Buchkapitel. Für einen Einblick in die Funktionsweise von manchen Spielen bleibt manchmal nicht mehr als eine Sammlung von Interviews, Videos und Vorträgen, die mal mehr mal weniger an Fachpublikum gerichtet sind. So sammelt sich bei der Recherche eine bunte Mischung an Quellen an, die Lösungen beschreiben aber oft auch Lücken lassen. Die meisten der Quellen beschreiben oft nur einen Aspekt (bspw. die Effizienz eines Algorithmus), behandeln in aber nicht in seiner Gesamtheit. Das Ziel dieser Arbeit soll sein, einen Ansatz möglichst ganzheitlich zu beleuchten.

Zu guter Letzt bin ich bei der Recherche auch auf Quellen gestoßen, die auf den ersten Blick den Kern der Arbeit genau treffen wie z.B. „Multiplayer Pursuit-Evasion Games in Three-Dimensional Flow Fields“. Es ist interessant zu sehen, dass Flow Fields auch in Bereichen der Robotik oder, sogar wie in diesem Fall, bei Luft und Wasserfahrzeugen Anwendung finden. Die Nützlichkeit dieser Gebiete für die Spiele-Entwicklung ist aber verschwindend gering [5].

1.3 TECHNISCHE DETAILS

Die Umsetzung dieser Arbeit erfolgte vollständig in der Unity Engine (Version 2020.3.27f1). Die Szene wurde mit Blender erstellt.

Ausgeführt wurden alle Tests innerhalb des Unity Editors und nicht in einem Standalone Build. Dies wirkt sich im Allgemeinen negativ auf die Performance aus.

Ausgeführt wurden die Tests auf einem Desktop PC in einer Windows 10 Umgebung. Die Grafikkarte ist ein Modell GTX 750 Ti der Firma Nvidia. Bei der CPU handelt es sich um das Modell Ryzen 5 1600 der Firma AMD mit 3.20 GHz.

Zum Programmieren wurde die IDE Visual Studio 2022 verwendet.

2 VEKTORFELD NAVIGATION

Für Anwendungen deren Laufzeit mit einer sehr hohen Anzahl an Agenten gut skaliert, ist es essentiell, die betroffenen Algorithmen so zu gestalten, dass das Ergebnis einer Berechnung von einer möglichst hohen Anzahl an Agenten genutzt werden kann. Das betrifft Bereiche wie

- Animation
- generelles Verhalten
- Kollisionsvermeidung und die
- Navigation.

In den meisten Spielen ist die Navigation kein für die Performance ausschlaggebender Teil der Software. Oft müssen die Agenten nur über kurze Distanzen navigieren, wenn der Spieler in ihre Nähe kommt. Die Anzahl der Agenten ist oft im einstelligen bis niedrigen zweistelligen Bereich, einige der Agenten bewegen sich vielleicht sogar auf vorgegebenen Pfaden und benötigen gar keine Möglichkeit zur Navigation. So können diese Spiele problemlos die Navigation einfach für jeden Agenten vollständig neu berechnen.

Ein Genre, bei dem das häufig anders ist, sind Echtzeit-Strategie-Spiele (RTS). Diese haben nicht selten eine hohe Anzahl an Agenten gleichzeitig zu steuern. Dabei können diese durchaus weite Distanzen zurücklegen, andere Agenten verfolgen und müssen auf Änderungen in der Umgebung eingehen.

Ein weit verbreitetes System in diesem Genre, um Pfade für Agenten zu berechnen, ist die Vektorfeld Navigation, auch „Flow Field Navigation“ oder „Dijkstra Map“ genannt. Auch in Teilen der Robotik und bei der Navigation von Drohnen kommt diese Art der Navigation zum Einsatz [5].

Das Prinzip ist auf den ersten Blick sehr einleuchtend. Die Agenten bewegen sich auf einem Feld (Graphen) dessen Knoten alle einen Vektor halten, der die Richtung angibt, in die sich ein Agent bewegen muss, um den Zielknoten zu erreichen. Zuerst wird nun ein Pathfinding-Algorithmus auf dem Graphen ausgeführt. Danach werden die berechneten Pfade in Form von Vektoren in die Knoten geschrieben. Das bedeutet, dass jeder Agent für jeden Tick, einfach den Vektor des Knotens, auf dem er sich befindet, abruft und so die Richtung erhält, in die er sich bewegen muss [1].

So kann das Ergebnis des Algorithmus mit minimalem Aufwand von einer hohen Anzahl an Agenten genutzt werden. Falls das Abrufen des Vektors durch direkten Zugriff auf den Graphen möglich ist, wächst der Aufwand für jeden Agenten lediglich um eine Konstante. Es bedarf keiner neuen Berechnungen, bis sich der Spieler bewegt.

Der Aufwand für den Pathfinding-Algorithmus wird zu einem späteren Zeitpunkt behandelt.

3 ERSTELLEN DES GRAPHEN

Die Vektorfeld Navigation, wie im vorherigen Abschnitt beschrieben, wird in den meisten Fällen, insbesondere bei den genannten Echtzeit-Strategie-Spielen auf ein zweidimensionales Spielfeld angewandt. Das hat einige Vorteile, die die Implementation erleichtern.

Zum einen kann das Spielfeld ganz intuitiv und ohne großen Aufwand in gleich große quadratische Zellen unterteilt werden, die dann die Knoten für den Graphen bilden (siehe Abbildung 1).

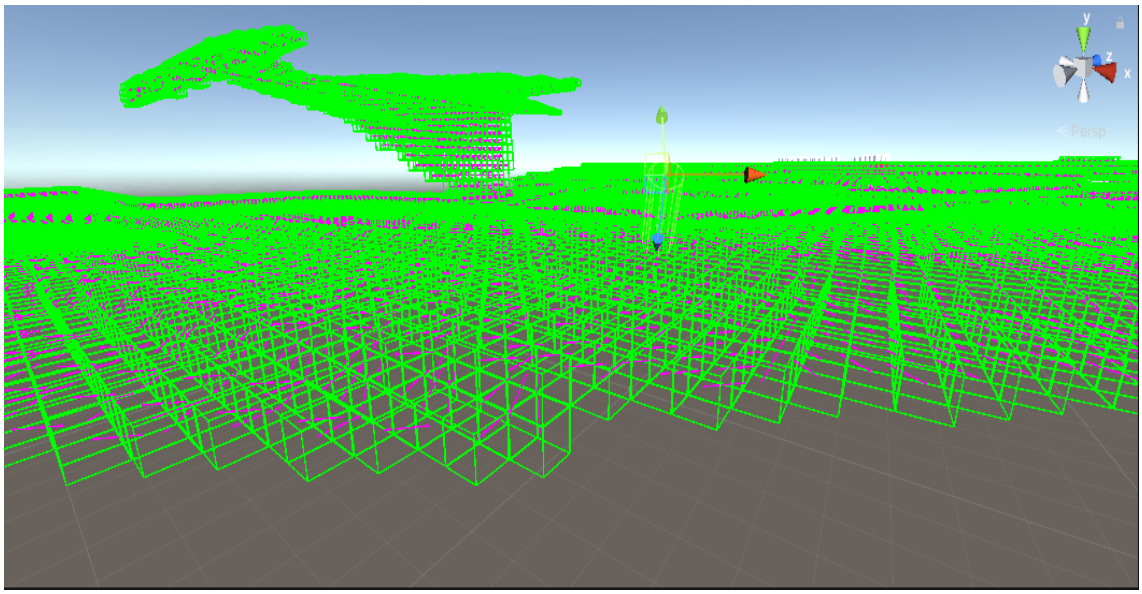


ABBILDUNG 1: DIE QUADRATISCHEN ZELLEN (EIGENE ABBILDUNG)

Auf diese Zellen kann dann während der Ausführung direkt und mit minimalem Aufwand zugegriffen werden. Es bedarf lediglich der Koordinaten des Agenten mit dem entsprechenden Vektor, der zur Navigation benötigt wird. Dieser kann sofort gefunden werden.

Für den Fall, der in dieser Arbeit betrachtet wird, also die Anwendung für eine komplexe dreidimensionale Szene, möchte man natürlich ebenso von der Effizienz dieses Ansatzes profitieren. Es bedarf allerdings eines anderen Ansatzes, um den Graphen selber zu erstellen. Schließlich ist nicht direkt offensichtlich, wie man für eine komplexe Geometrie, einen Graphen erstellt, der für die Navigation verwendet werden kann.

Ein möglicher Ansatz, ist es, manuell bei der Erstellung der Geometrie gleich auch den Graphen anzulegen. Dafür kann beispielsweise eine zusätzliche unsichtbare Geometrie, die auf der begehbaren Oberfläche der Szene liegt, angelegt werden, die dann als Grundlage für den Graphen verwendet wird. Dieser Ansatz ist mit erheblich mehr Arbeitsaufwand verbunden und deswegen wohl keine optimale Lösung. Denkbar ist auch ein Verfahren, das über Mapping

arbeitet. Bei der Erstellung der Geometrie kann gleichzeitig noch eine Map mit einem Raster angelegt werden, das die Oberfläche der Geometrie in Zellen unterteilt. Hier ist es allerdings nicht klar, wie die Position der Agenten in der Szene, zu der entsprechenden Zelle (Knoten) gemappt werden kann.

Der gängige Ansatz in den etablierten Game Engines sind die sog. „Navigation Meshes“. Bei diesem Verfahren wird ein Mesh aus Dreiecken erstellt, indem errechnet wird, an welchen Stellen auf der Oberfläche der Geometrie ein Agent einer bestimmten Größe Platz hat und die Neigung der Oberfläche sich in einem gewissen Rahmen hält. [Q] Auf dem sich daraus ergebenden Graphen wird dann ein A* Algorithmus ausgeführt.

Auf den ersten Blick erscheint es logisch, diesen Graphen als Grundlage für das Vektorfeld heranzuziehen, jedoch ergeben sich dabei einige Probleme: Zum einen, haben die Dreiecke aus denen das Mesh besteht unterschiedliche Größen und Ausdehnungen (siehe Abbildung 2).

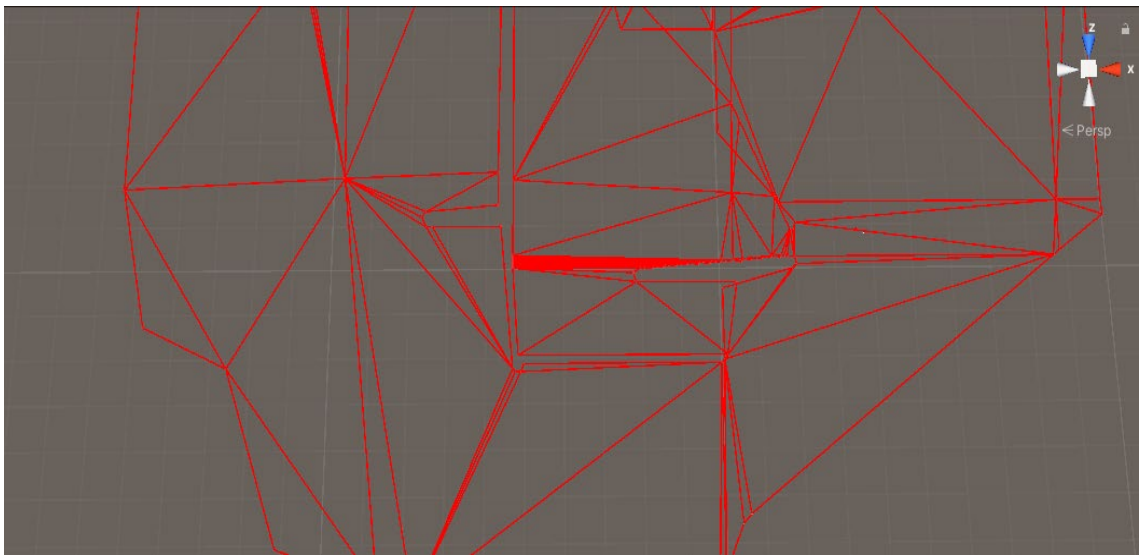


ABBILDUNG 2: DREIECKE DES NAV-MESH MIT UNTERSCHIEDLICHEN GRÖßEN (EIGENEN ABBILDUNG)

Die gängigen Navigations-Algorithmen laufen auf einem Graphen, dessen Knoten natürlich keine geometrische Ausdehnung besitzen. Es muss also ein Punkt festgelegt werden, der das Dreieck als Knoten des Graphen repräsentiert. Je nach spezifischer Form des Meshes kann dies dazu führen, dass nicht mehr der optimale Pfad gefunden wird, weil der Knotenpunkt des Dreiecks zu weit abseits des optimalen Pfades liegt, dieser aber eigentlich das besagte Dreieck durchqueren müsste um optimal zu sein.

Ein weiteres Problem ist, dass wenn das gesamte Dreieck nur einen einzigen Vektor bekommt, kann die daraus resultierende Bewegung des Agenten den angedachten Knoten auf dem optimalen Pfad verfehlen. Abhängig von Ausdehnung des Dreiecks und der Position des Agenten kann diese Abweichung sehr signifikant ausfallen.

Für die geometrische Unterteilung der Szenen-Oberfläche als Grundlage für eine Vektorraum Navigation ist es also sehr vorteilhaft, eine Einteilung in kleine und uniforme Einheiten vorzunehmen. Durch diesen Ansatz werden die oben genannten Probleme mitigiert. Die Abweichung der Bewegung des Agenten vom optimalen Pfad wird durch eine angemessen klein gewählte Größe der Zelle vernachlässigbar. Es kann allerdings immer noch nicht garantiert werden, dass der Pfad, den beispielsweise ein A* Algorithmus findet, optimal ist. Dies liegt daran, dass Pathfinding-Algorithmen auf einem Graphen mit gewichteten Kanten operieren.

Für Anwendungen, die sich auf eine konkrete geometrische Grundlage beziehen (bspw. Szene-Geometrie, Straßennetz), wird die Distanz zwischen den Knoten als Gewichtung der entsprechenden Kante genutzt.

Haben die Knoten nun aber wie in diesem Fall eine geometrische Ausdehnung, kann das Kantengewicht lediglich die Distanz zwischen exakt zwei Punkten innerhalb des einen und des anderen Knotens ausdrücken. Es ist dementsprechend also immer möglich, dass ein optimaler Pfad zwar durch einen Knoten führen müsste, diesen aber meidet, da die Entfernung zwischen den Punkten, die diese benachbarten Knoten repräsentieren größer ist, als die Entfernung zum Punkt, der einen anderen Knoten repräsentiert. Dieses Problem wird allerdings auch vernachlässigbar, wenn die Geometrie des Knoten klein gewählt wird, da etwaige Abweichung seltener werden und außerdem falls sie dennoch eintreten, nur einen geringen Umweg bedeuten, der in der realen Anwendung nicht auffallen sollte.

Das Ziel ist also, die Oberfläche der Geometrie in uniforme Abschnitte zu unterteilen, mittels der bereits weit verbreiteten Methode der Nav-Meshes. Dafür könnte man versuchen, die Dreiecke des Nav-Meshes zu transformieren.

Vielversprechender ist allerdings der Ansatz, die Szene in gleichgroße Würfel zu unterteilen und dann herauszufinden, welche dieser Abschnitte sich mit den Dreiecken des Nav-Meshes schneidet. Somit erhält man eine Menge an uniformen Zellen, die die Oberfläche des Levels repräsentieren. Diese kann dann ähnlich dem Nav-Mesh bei der Erstellung der Szene berechnet und bei der Ausführung geladen werden. Dieses Vorgehen hat den zusätzlichen Vorteil, dass wenn die Position des Agenten im World-Space gegeben ist, es einen direkten Zugriff auf die jeweilige Zelle ermöglicht, in der sich der Agent befindet. Dies ist vorteilhaft, da die Abfrage der Bewegungsrichtung für einen Agenten, über die Zelle, in welcher dieser sich befindet, während der Laufzeit sehr häufig für jeden der Agenten durchgeführt werden muss.

Wichtig dabei ist, die Größe der Zellen passend zu wählen. Ist diese zu groß, kann es passieren, dass zwei Nav-Mesh Dreiecke, die keine direkte Verbindung haben (bspw. weil sie auf unterschiedlichen Seiten einer Wand sind), in der gleichen Zelle liegen (siehe Abbildung 3).

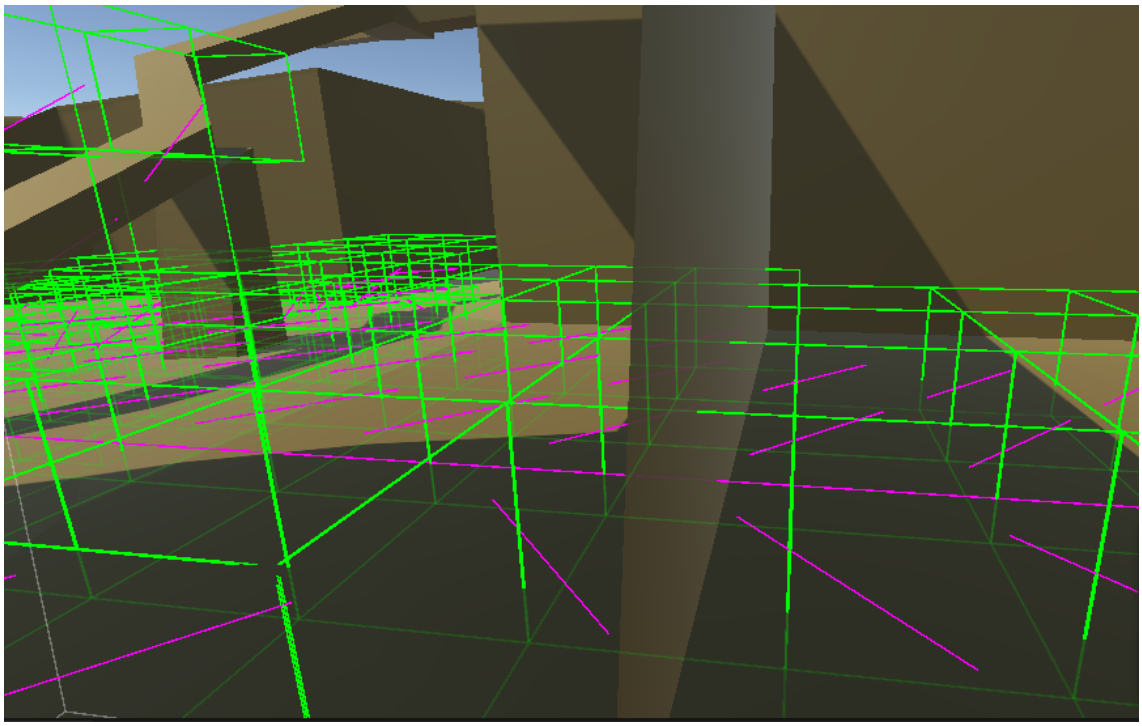


Abbildung 3: Beide Seiten der Wand liegen in der gleichen Zelle (eigenen Abbildung)

Dadurch würde ein Pathfinding-Algorithmus einen Pfad durch diese Wand planen, dem die Agenten nicht folgen können. Ein ähnliches Problem kann entstehen, wenn vertikale Abstände zwischen Oberflächen zu gering sind. Deswegen ist es wichtig sich für die Größe der Zelle an Radius und Höhe des Agenten, der zur Berechnung des Nav-Meshes benutzt wurde, zu orientieren. Besonders ist dabei auf den Radius zu achten, denn dieser reguliert den Abstand, den das Nav-Mesh zu Wänden und anderen Hindernissen einhält. Somit kann das eben beschriebene Problem der Überschneidungen durch Wände verhindert werden.

Die Größe der Zelle g möglichst klein zu wählen ist allerdings auch keine Lösung. Die Anzahl der Zellen ist $\frac{1}{g^2}$ und nimmt damit asymptotisch zu für $g \to 0$. Der Aufwand für den A* Algorithmus steigt mit der Anzahl der Knoten. Außerdem ist es bei einer solchen Menge an Knoten sehr schnell möglich, auf Speicherplatzprobleme zu stoßen.

Wegen dieses Problems ist es ebenfalls notwendig, dass die Größe der Szene, die zu jedem gegebenen Zeitpunkt geladen wird, nicht zu groß ist. Es ist also entscheidend, die Größe der Zelle passend zu wählen. Leider kann diese nur vor der Laufzeit ausgewählt werden, da die Erstellung des Zellen-Netzes eine relevante Zeit in Anspruch nimmt (Siehe hierzu Kapitel 5 „Performance“).

3.1 INTERSECTION TEST

Bleibt aber immer noch die Frage, wie aus einem Mesh aus Dreiecken ein Netz aus Zellen erstellt werden kann. Dafür muss ein geometrischer Intersection Test durchgeführt werden, um festzustellen, ob eine gegebene Zelle und ein gegebenes Dreieck sich überschneiden. Dabei ist es selbstverständlich nicht möglich, jede Zelle mit jedem Dreieck zu vergleichen. Die nicht übermäßig große Szene, die für Testzwecke verwendet wurde, hat ~300 Dreiecke und ~80.000 Zellen. Die daraus resultierenden 24 Millionen nötigen Tests, sind selbst für eine Berechnung, die vor der Laufzeit ausgeführt werden kann, nicht praktikabel.

Es ist also nötig, die durchgeführten Tests zu reduzieren. Dafür kann man sich die gleichmäßige Anordnung der Zellen zunutze machen. Hierfür muss man für jedes Dreieck $D = \{p_1, p_2, p_3\}$ den Punkt

$$p_{min} = \begin{pmatrix} x_1 \\ y_1 \\ z_1 \end{pmatrix}$$

ermitteln, mit:

$$X_D = \left\{x_{p_1}, x_{p_2}, x_{p_3}\right\}, Y_D = \left\{y_{p_1}, y_{p_2}, y_{p_3}\right\}, Z_D = \left\{z_{p_1}, z_{p_2}, z_{p_3}\right\}$$

für den gilt:

$$x_1 \in X_D \mid \nexists\, x \in X_D \rightarrow x_1 > x \qquad y_1 \in Y_D \mid \nexists\, y \in Y_D \rightarrow y_1 > y$$

$$z_1 \in Z_D \mid \nexists\, z \in Z_D \rightarrow z_1 > z$$

Und den Punkt $p_{max} = \begin{pmatrix} x_2 \\ y_2 \\ z_2 \end{pmatrix}$ für den gilt:

$$x_2 \in X_D \mid \nexists\, x \in X_D \rightarrow x_2 < x \qquad y_2 \in Y_D \mid \nexists\, y \in Y_D \rightarrow y_2 < y$$

$$z_2 \in Z_D \mid \nexists\, z \in Z_D \rightarrow z_2 < z$$

Mit diesen beiden Punkten können nun aus allen Zellen

$$Z = \{z_{x,y,z} \mid 0 \leq x < x_{max}, 0 \leq y < y_{max}, 0 \leq z < z_{max}\},$$

jene bestimmt werden, die möglicherweise eine Überschneidung mit D haben, nämlich alle Zellen deren Mitte innerhalb des von p_{max} und p_{min} aufgespannten Quaders Q liegen

$$Q = \{z_{x,y,z} \in Z \mid x_2 * cellSize < x * cellSize < x_1 * cellSize, y_2 * cellSize < y * cellSize < y_1 * cellSize, z_2 * cellSize < z * cellSize < z_1 * cellSize\}.$$

Zusätzlich alle Zellen $\in Z$ deren Indizes exakt um eins größer oder kleiner sind als die minimalen und maximalen Indizes in Q. Die Menge

$$Q \cup \{\{z_{x,y,z} \in Z \mid x = x_{Qmin} - 1 \vee x = x_{Qmax} + 1 \vee y = y_{Qmin} - 1 \vee y = y_{Qmax} + 1 \vee z = z_{Qmin} - 1 \vee z = z_{Qmax} + 1\}$$

repräsentiert also alle Zellen, für die eine Überscheidung mit D überprüft werden muss.

Hierfür eignet sich der von Tomas Akenine-Möller 2001 in seinem Paper: „Fast 3D Triangle-Box Overlap Testing“ beschriebene Ansatz. Dieser berechnet spezifisch nur die Überlappung eines Dreiecks mit einem Würfel und ist damit besonders effizient für diesen Fall. Der Autor erwähnt auch eine Verwendung im Bereich des Echtzeit-3D-Renderns, was weit über die Laufzeitanforderungen dieser Applikation hinausgeht. [4]

Das Verfahren spezialisiert sich im Weiteren auf „axis-alligned bounding boxes“ (AABB), also Quader deren Kanten parallel zu den Achsen des Welt-Koordinatensystems verlaufen. Unsere Zellen, die wir überprüfen wollen, erfüllen diese Eigenschaft, was die Berechnung weiter vereinfacht.

Der Ansatz basiert auf dem „seperating-axis-theorem“, welches besagt, dass zwei konvexe Polyeder, genau dann disjunkt sind, wenn sie entweder anhand einer zur Normalen der Fläche der Polyeder parallelen Achse voneinander getrennt werden können oder durch eine Achse, die das Resultat des Kreuzprodukts einer Kante des einen und des anderen Polyeders sind. [4]

Dieses Verfahren verschiebt die Bounding-Box und das Dreieck so, dass die Bounding Box im Ursprung des Koordinatensystems liegt. So können die Vektoren (0,0,1), (0,1,0), (1,0,0) als Normale der Box verwendet werden.

Im ersten Schritt wird dann die minimale AABB für das Dreieck berechnet und diese mit der ursprünglichen AABB verglichen [4], ähnlich dem vorangegangen Verfahren zu Selektion der Zellen. Dafür wird das maximale/minimale Skalarprodukt für diese Normalen mit den Eckpunkten des Dreiecks ermittelt. Ist eines dieser Skalare kleiner/größer als die maximale/minimale Ausdehnung der AABB entlang dieser Achse, kann es keine Überschneidung geben und der Algorithmus terminiert. Für die Normale (1,0,0) bedeutet das also, dass es keine Überscheidung geben kann, wenn:

$$\exists\, x \in D \mid (1,0,0) \cdot \boldsymbol{x} < \boldsymbol{x_{D_{min}}} \quad \text{oder} \quad \exists\, x \in D \mid (1,0,0) \cdot \boldsymbol{x} > \boldsymbol{x_{D_{max}}}$$

Wenn diese Bedingung für alle 3 Normalen nicht erfüllt ist, geht man zum nächsten Test über. Bei diesem wird die Überschneidung der Ebene, in der das Dreieck liegt, mit der AABB überprüft. Dafür werden die zwei Punkte herangezogen, deren Richtung am meisten mit der Normalen des Dreiecks übereinstimmt [4]. Die zwei Punkte deren Skalarprodukt mit der Normalen n des Dreiecks also minimal/maximal sind, werden ermittelt und danach auf folgende Bedingung überprüft für $a \in D$:

$$(x \cdot \boldsymbol{n_D})_{min} < \boldsymbol{n} \cdot \boldsymbol{a} \text{ oder } (x \cdot \boldsymbol{n_D})_{max} > \boldsymbol{n} \cdot \boldsymbol{a}$$

Sind alle 4 vorangegangen Test negativ ausgefallen, ist im letzten Schritt eine weitere Bedingung zu überprüfen. Dieser wird jeweils für alle möglichen Kombinationen von Normalen der AABB mit Vertices des Dreiecks durchgeführt, also insgesamt 9-mal. Für diesen wird zuerst der Vektor v ermittelt mit $v = x \times \boldsymbol{n}$ mit $\boldsymbol{x} \in$ D und $n \in \{(1,0,0), (0,1,0), (0,0,1)\}$. Danach wird das maximale/minimale Skalarprodukt d_{min}/d_{max} von v mit den Punkten des Dreiecks, ebenso wie mit den Eckpunkten der AABB a_{min}/a_{max} , ermittelt. Diese werden dann auf folgende Bedingungen überprüft:

$$a_{min} < d_{min} \text{ oder } a_{max} > d_{max}$$

Sind auch diese 9 Test alle negativ, wissen wir, dass das Dreieck und die Bounding-Box disjunkt sind. Die Zelle hat also keine Überschneidung mit der Oberfläche der Szenen-Geometrie, auf der sich die Agenten bewegen und soll somit nicht in die Menge der Zellen aufgenommen werden, die für die Vektorfeld Navigation verwendet werden. Wenn der Prozess für alle Dreiecke des Nav-Meshes und alle potentiell überschneidenden Zellen durchgeführt wurde, erhalten wir das fertige Feld, auf dem die Navigation stattfindet. Wie dieses beispielhaft aussehen kann, ist in Abbildung 4 zu sehen.

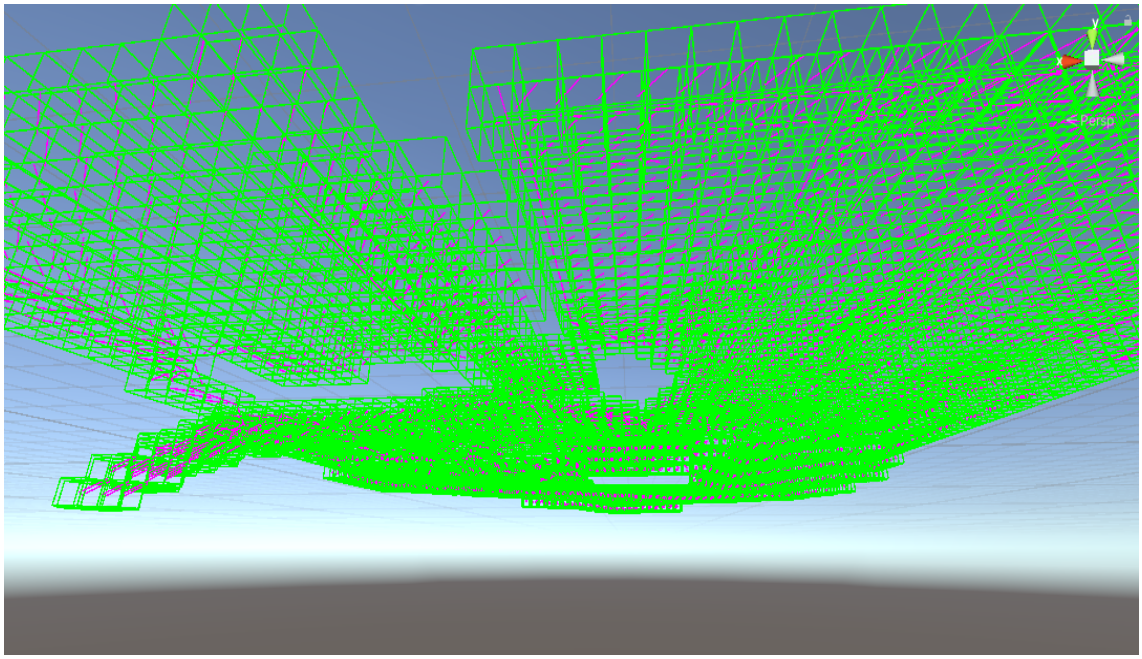


ABBILDUNG 4: DAS FERTIGE NETZ DER ZELLEN (EIGENE ABBILDUNG)

3.2 IMPLEMENTIERUNG

Jetzt wissen wir, wie man den Graphen erstellt, indem man bestimmt, welche Zellen auf der Oberfläche der Szene liegen. Allerdings ist dieser Prozess alles andere als echtzeitfähig, schon gar nicht für eine Anwendung, die mit hunderten bis tausenden Agenten funktionieren soll. Die folgende Tabelle zeigt, wie lange die Berechnung für verschieden Parameter dauert.

Es ist damit klar, dass dieser Prozess vor der Ausführung abgeschlossen sein muss. Auch eine Berechnung beim Laden der Szene fällt weg. Bei Szenen, die statisch sind, bei denen also keine Veränderungen der begehbaren Flächen während dem Gameplay stattfinden, ist dies ohnehin das logische Vorgehen. Auf dynamische Szenen wird zu einer späteren Zeit eingegangen. Eine Datenstruktur, die nur eine minimale Anzahl an Informationen speichert, kann wie folgt aussehen:

```
public class SeriField
{
public float originX;
public float originY;
public float originZ;
public List<int> x = new List<int>();
public List<int> y = new List<int>();
public List<int> z = new List<int>();
public List<int> portalIndex = new List<int>();
}
```

Diese Struktur kann während der Entwicklung des Spiels gefüllt und dann beim Laden der Szene ausgelesen werden. Es braucht lediglich den Koordinaten-Ursprung für das Feld der Zellen, die Indizes aller aktiven Zellen und die Indizes aller Zellen, die zu einem gegebenen Portal gehören.

Außerdem sind die ersten Einträge der Listen x, y und z die Dimension des Feldes, das für die Zellen angelegt werden muss. Sollte die Größe der Zelle nicht gleich für alle Szenen sein, muss diese auch hinterlegt werden. So kann die Szene nun ohne große Berechnungen geladen werden.

3.2.1 DYNAMISCHE SZENEN

Zusätzliche Betrachtungen sind notwendig, wenn man es mit einer dynamischen Szene zu tun hat, also einer Szene, bei der sich während der Laufzeit der begehbare Bereich ändert. In vielen Spielen passiert dies regelmäßig, entweder durch Handlungen des Spielers, der KI-Agenten oder das Voranschreiten des Spielgeschehens. Diese Veränderung des Vektorfelds lässt sich in zwei verschieden Kategorien unterteilen:

1. **Deterministische Veränderung**,
   die immer nach dem gleichen Muster geschehen. Beispielsweise eine Wand, die von den Zombies, die den Spieler jagen, durchbrochen werden kann oder ein umfallender Baum, der einen Teil der Karte verschließt.

2. **Nicht-Deterministische Veränderung**,
   die nicht vorbestimmt sind und bei jeder Ausführung anders ausfallen können. Beispielsweise sind dies Explosionen, die vom Spieler ausgelöst werden und die Geometrie der Szene verändern können.

Der erste Fall, die deterministische Veränderung, ist bei weitem die einfachere Variante. Für vorbestimmte Änderungen der Zellen kann der alternative Zustand schon im Voraus berechnet werden und dann während der Laufzeit auf das aktuelle Feld übertragen werden. Bei der Entwicklung des Spiels kann dann auch die Szene in ihrem alternativen Zustand kalkuliert werden. Dann werden alle Zellen, die sich vom ursprünglichen Zustand unterscheiden, in einer Datenstruktur ähnlich der aus dem vorangegangenen Kapitel gespeichert.

Tritt dann die Transformation während des Spiels ein, werden einfach diese Änderungen angewendet. Im gleichen Schritt sollte auch eine Neuberechnung des Pathfinding ausgelöst werden damit keiner der Agenten durch eine nun nicht mehr zugängliche Zelle versucht zu navigieren. Dieser Vorgang kann mit geringem Aufwand während der Laufzeit ausgeführt werden.

Anders sieht es für Nicht-Deterministische Veränderungen aus. Diese können nicht vor der Ausführung berechnet werden. Es müssen also folgende Schritte durchgeführt werden, um die Zellen anzupassen:

1. Neuberechnung des Nav-Meshes.
   Am besten nur für den Teil der Szene, der von der Veränderung betroffen ist, falls dies möglich ist.
2. Selektieren der Zellen, die sich von der ursprünglichen Szene unterscheiden, mittels Intersection-Test.
3. Übertragen der Änderungen auf das aktuelle Feld.

Ja nach Aufbau der Szene kann es sein, dass für Schritt 1 das Nav-Mesh für die gesamte Szene berechnet werden muss. Dieser Prozess ist nicht geeignet, ihn während der Laufzeit auszuführen. Schritt 2 kann mit wenig Aufwand verbunden sein, gegeben die von der Veränderung betroffen Dreiecke sind bekannt. Es ist allerdings nicht klar, dass diese Veränderungen bekannt sind, da sich die Anzahl der Dreiecke ebenso wie deren Ausdehnung vollkommen unberechenbar verändern. Schritt 3 stellt, ähnlich wie bei den Deterministischen Veränderungen, kein Problem dar.

Damit der gegebene Ansatz also für dieses Art der Veränderung funktioniert, muss die Anzahl der notwendigen Berechnungen stark eingeschränkt werden können. Dafür gibt es keinen gängigen Ansatz. Es ist auch anzumerken, dass selbst im Optimalfall die Operationen 1 und 2 noch keine Laufzeit-Optimierten-Berechnungen sind. Sollte in einem gegebenen Spiel also viele unberechenbare Veränderungen der Szene auftreten, ist es ratsam einen anderen Ansatz zur Erstellung des Vektorfelds zu wählen.

4 PATHFINDING ALGORITHMUS

4.1 A* ALGORITHMUS

4.1.1 EIGNUNG

Der mit Abstand relevanteste Algorithmus für Anwendungen, die effiziente Pfade durch einen Graphen mit gewichteten Kanten berechnen ist der A* Algorithmus. Basierend auf dem Dijkstra-Algorithmus, welcher den Graphen vollständig löst und somit den optimalen Pfad zu allen (erreichbaren) Knoten des Graphen berechnet, endet A* nachdem der Optimale Pfad zu einem gegebenen Zielknoten ***z*** gefunden wurde. Dabei bedient er sich einer Heuristik ***h(x)***, die für jeden Knoten x einen Wert annimmt, der eine Schätzung für die Distanz zum Zielknoten ist. Dabei darf sie aber niemals kleiner sein als die tatsächlich optimale Distanz auf dem Graphen [2].

Dadurch hat A* zwei Eigenschaften, die ihn zur optimalen Lösung für unzählige Anwendungen machen: Der Graph wird nur so weit gelöst wie nötig und dabei wird die Berechnung durch die Heuristik in die „richtige“ Richtung gelenkt, um die Zahl der Knoten, die beachtet werden, zu minimieren.

Dabei ist A* optimal und im allgemeinen Fall optimal effizient [2]. Es ist also wenig verwunderlich wie weit verbreitet A* und seine Varianten sind, insbesondere im Bereich der Spieleentwicklung. So nutzt beispielsweise auch die Nav-Mesh Navigation der Unity Engine, dessen Mesh die Grundlage für den Graphen der in dieser Arbeit verwendet wurde, die A* Navigation [6].

4.1.2 HEURISTIK

Der Gedanke ist also sehr naheliegend eine Version des A* Algorithmus zu nutzen, um den Pfad durch den Graphen zu berechnen und dann mit diesem Ergebnis das Vektorfeld zu bestücken auf dem sich die Agenten schlussendlich bewegen werden.

Dabei sei die Menge X die Menge aller Knoten, über die die Agenten navigieren, also spezifisch alle dreidimensionalen Würfel, die im Voraus als auf der navigierbaren Oberfläche der Szene-Geometrie liegend bestimmt wurden. Die Menge K sei die Menge aller Kanten des Graphen wobei $k_{x_1,x_2} \in K$ genau dann gilt wenn $x_1$und x_2 benachbart sind. In diesem Fall bedeutet dies, dass $x_1$und x_2 mindestens einen identischen Eckpunkt besitzen:

$$k_{x_1,x_2} \in K \rightarrow \exists \vec{v}_1 \in x_1, \vec{v}_2 \in x_2 : \vec{v}_1 = \vec{v}_2$$

Für die Heuristik *h(x)* wird üblicherweise die direkte Distanz $|\vec{x} - \vec{t}|$ zwischen Knoten x zum Zielknoten *t* herangezogen, da diese nicht nur sehr gut darin ist, die Anzahl der zu betrachtenden Knoten zu minimieren, sondern auch weil auf dem Graphen kein kürzerer Weg

zwischen *x* und *t* gefunden werden kann. Ist diese Eigenschaft nicht gegeben, kann ein kürzer Pfad auf dem Graphen gefunden werden als die Heuristik suggeriert. Somit ist das Ergebnis von A* nicht mehr optimal [2].

Auf die Art der Heuristik wird zu einem späteren Zeitpunkt noch eingegangen. Zunächst lässt sich aber feststellen, dass sowohl diese Heuristik als auch der Algorithmus nur jeweils ein Ziel sowie Startknoten annehmen.

Die in dieser Arbeit betrachtete Anwendung hat zwar auf der einen Seite nur den Spieler, aber auf der anderen mehrere Agenten. Wir müssen also eine modifizierte Version von $A^*(G, s, t, h(x))$ heranziehen, die eine Menge von Zielknoten T oder eine Menge an Startknoten S akzeptiert.

Zunächst muss die Frage geklärt werden, ob der Spieler als Ziel oder als Startknoten eingesetzt werden sollte. A* berechnet nicht nur den optimalen Pfad zum Zielknoten, sondern auch zu allen Knoten auf diesem Pfad und zu allen Knoten, die während der Laufzeit traversiert wurden. Für letztere ist der Pfad zwar nicht zwingend optimal, aber in einem Graphen, in dem alle Kanten identisch gewichtet sind (alle Zellen haben den gleichen Abstand zueinander), ist das Ergebnis akzeptabel.

Somit ergibt sich ein Feld, das vom Startknoten aus zu einer bestimmten Menge an Knoten einen effizienten Pfad enthält (siehe Abbildung 5).

Setzt man, wie auf den ersten Blick naheliegend, den Agenten als Start und den Spieler als Ziel, so kann man den Algorithmus für jeden Agenten ausführen und dann terminieren, wenn entweder der Zielknoten oder ein Knoten, der auf dem optimalen Pfad eines anderen Agenten liegt, erreicht wurde. Das ist intuitiv und effizient. Schließlich wird A* für jeden Agenten einer Gruppe, die nah zusammen stehen, sehr schnell terminieren, da er sofort auf den Pfad eines anderen Agenten trifft.

Wir können dann die optimalen Pfade der einzelnen Agenten in einem gemeinsamen Vektorfeld kombinieren und erhalten ein Feld, das nur in den Zellen der optimalen Pfade brauchbare Vektoren (nämlich die, die zum Ziel führen) enthält. Setzen wir allerdings den Spieler als Start des Algorithmus und den Agenten als Ziel und invertieren dann die Pfade des Ergebnisses, erhalten wir ein Feld in dem von allen Felder, die vom Algorithmus erreicht wurden, ein Pfad zur Position des Spielers führt.

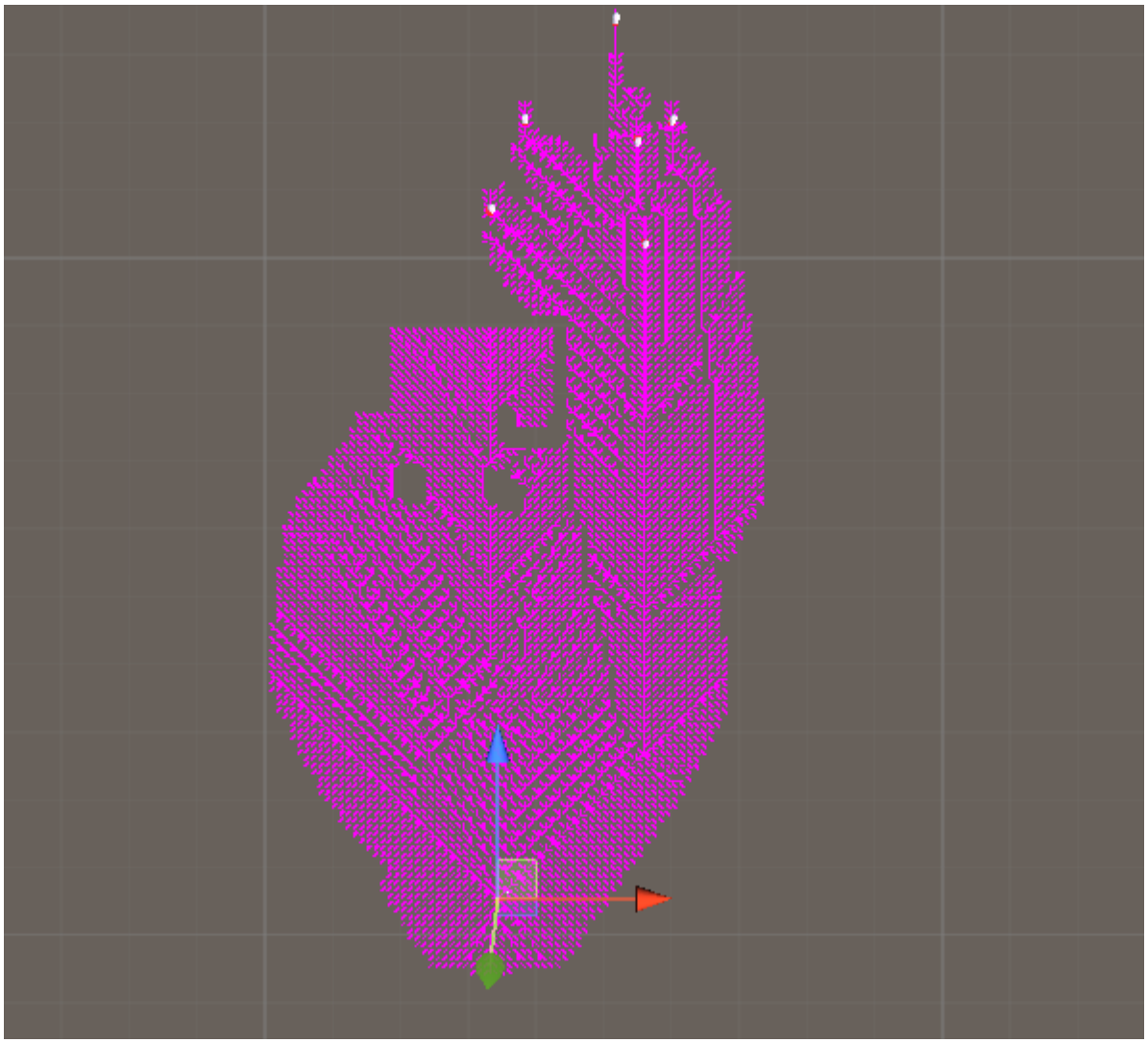


ABBILDUNG 5 EIN VEKTORFELD, DASS ZU ALLEN AGENTEN EINEN BEINAHE OPTIMALEN PFAD BERECHNET HAT (EIGENEN ABBILDUNG)

Das hat für die reale Anwendung in einem Spiel einige Vorteile. Sollten die Agenten während der Navigation vom optimalen Pfad abkommen, durch Kollisionsvermeidung, Spielgeschehen etc. hat der Knoten, auf dem sich der Agent befindet noch keinen gültigen Vektor zum Spieler. Somit müsste in diesem Fall A* erneut bemüht werden. Dieser sollte meistens sehr schnell terminieren, da er es nicht weit hat, um wieder auf den Optimalen Pfad zu stoßen.

Für die Laufzeit ist dieses Verhalten trotzdem von Nachteil, da die meisten dieser Knoten bereits während der vorherigen Ausführung traversiert wurden, dieses Ergebnis aber verworfen wurde, da der Knoten nur Teil eines Pfades war, der diesen Knoten mit dem Agenten verbindet und

nicht auf dem optimalen Pfad lag. Es ist also vorteilhaft, den Spieler als Startknoten zu setzen und dann für das Vektorfeld die Pfade in umgekehrter Reihenfolge zu durchwandern.

Das Ergebnis sieht dann wie in Abbildung 6 aus.

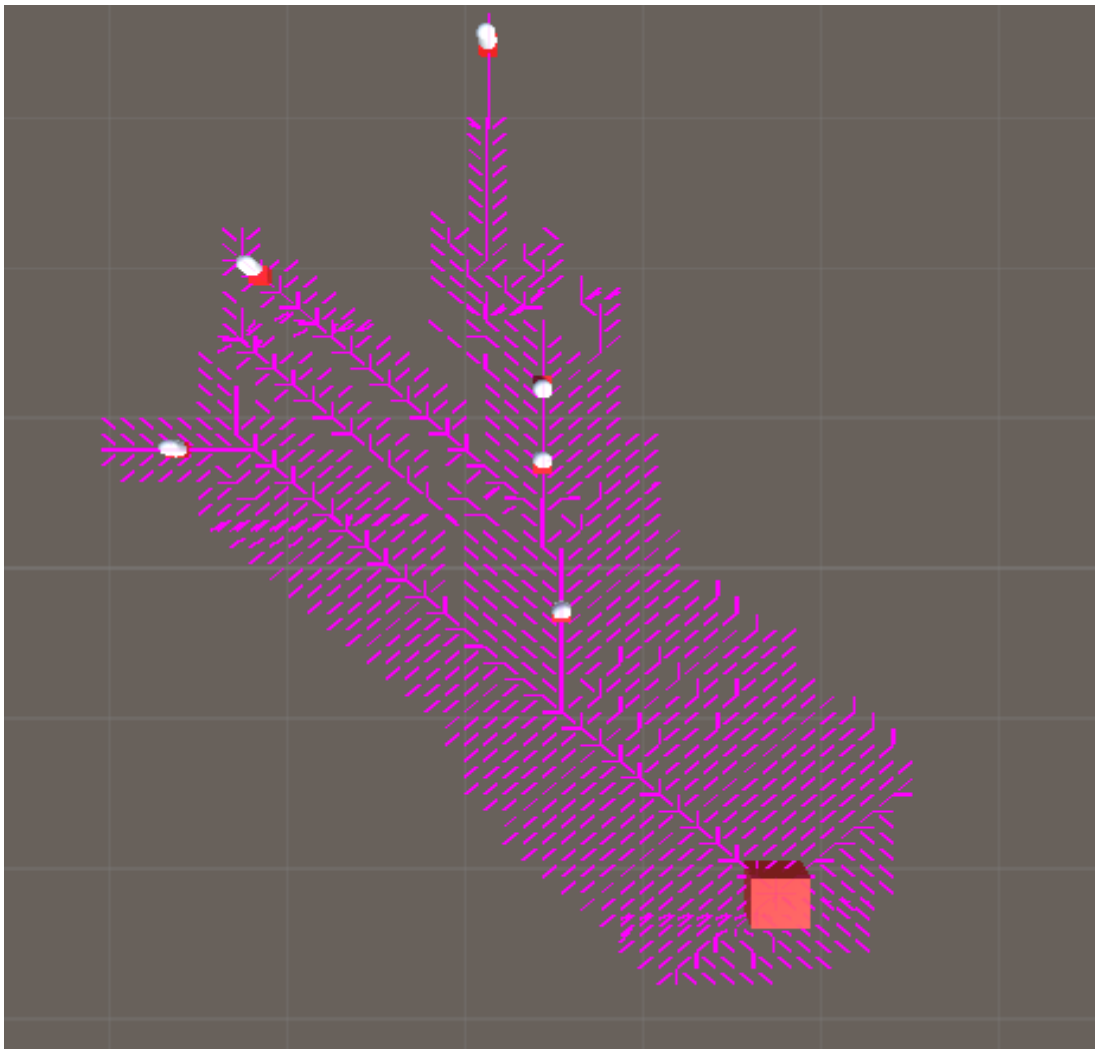


ABBILDUNG 6: DAS ERGEBNIS VON A*, WENN DER SPIELER (ROTES QUADRAT) ALS STARTPUNKT GESETZT WIRD. ALLE ERREICHTEN ZELLEN ENTHALTEN EINEN VEKTOR DER ZUM SPIELER FÜHRT (EIGENE ABBILDUNG)

Es bleibt allerdings wieder das Problem, dass es mehr als einen Zielknoten gibt. Ein Ansatz ist, die Heuristik $h_{t_1}(x)$ zu verwenden, wobei $t_1 \in T$ der Agent mit der geringsten direkten Distanz zum Spieler ist und $h_{t_1}(x)$ die direkte Distanz von Knoten x zum Startknoten s.

Wird ein Knoten $t \in T$ erreicht wird er aus T entfernt. Wurde der Knoten t_1 erreicht wird A* weiter ausgeführt nun aber mit der Heuristik $h_{t_y}(x)$, wobei $t_y \in T$ der Knoten mit der geringsten direkten Distanz zum Spieler ist.

Wurden alle Agenten erreicht ($T = \emptyset$), terminiert die Ausführung. Das Ergebnis ist ein garantiert optimaler Pfad zu t_1. Dies gilt nicht für die anderen Agenten.

Außerdem werden alle Agenten die hinter t_1 liegen dazu tendieren, zuerst in Richtung Knoten t_1 zu navigieren. Somit werden alle Agenten, die aus einer ähnlichen Richtung kommen schnell auf einen gleichen Pfad stoßen. Dieses Verhalten kann je nach erwartetem Verhalten der Agenten erwünscht oder unerwünscht sein.

Im Allgemeinen weicht das Ergebnis für einzelne Agenten nicht stark von der Ausführung von A* mit nur einem Ziel ab. (Siehe Abbildung 7 und Abbildung 8)

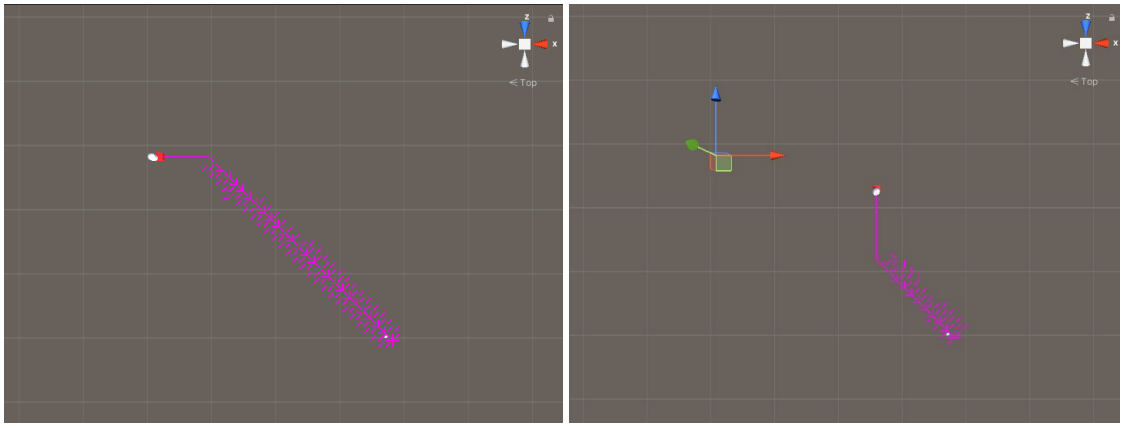


Abbildung 7: Das Ergebnis für eine Heuristik mit nur einem Ziel (Eigene Abbildung)

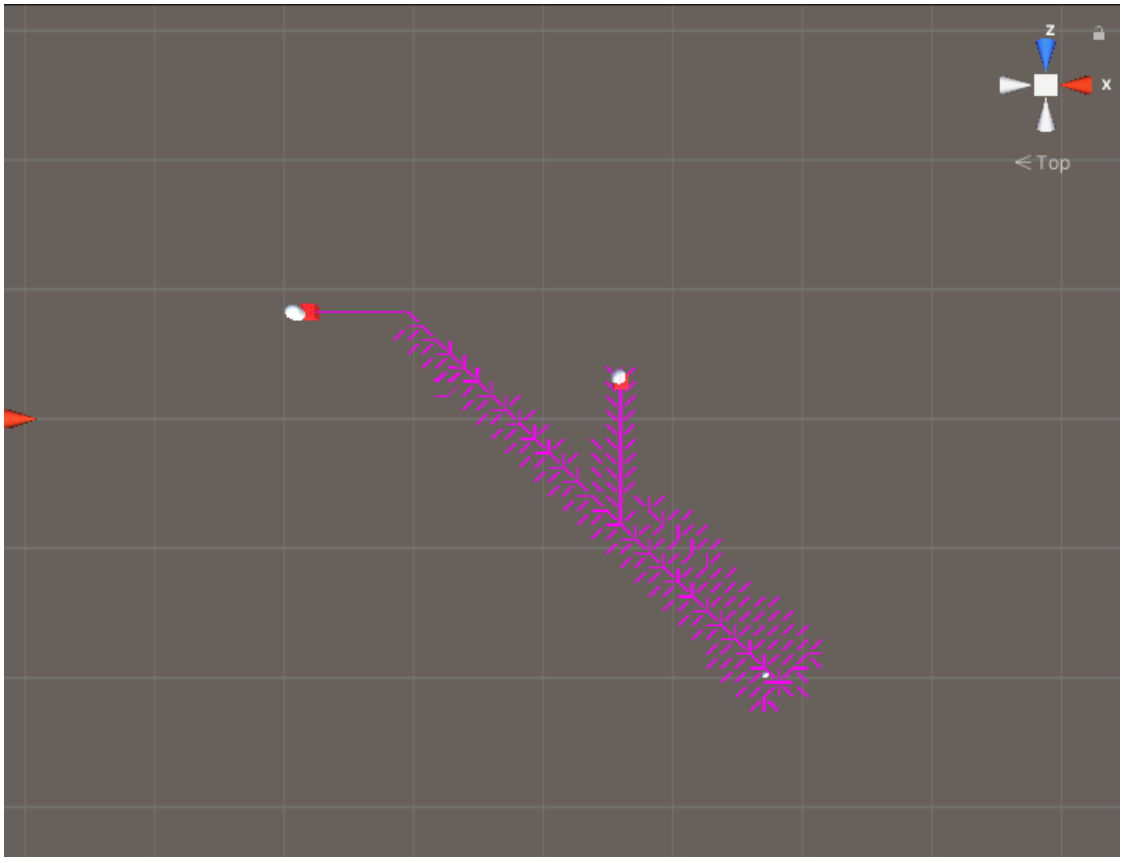


Abbildung 8: Das Ergebnis einer Heuristik mit zwei Zielen. (Eigene Abbildung)

4.1.3 WAVE FRONT

Ein weiterer elementarer Bestandteil jedes Pathfinding Algorithmus ist die sogenannte „Wave front“. Die Wave front ist eine Bezeichnung für das Verhalten eines Pathfinding Algorithmus, da dieser sich „wie eine Welle“ über einen Graphen ausbreitet [1].

Gemeint ist damit, dass der Algorithmus immer eine Menge an Zellen hat, die er in der nächsten Iteration betrachtet. Die Wave front ist also die Menge W an Knoten, die mit einem der Knoten aus der Menge der bereits besuchten Knoten S benachbart sind, selbst aber nicht Teil von W sind.

Sei K die Menge aller Knoten und N(x) die Menge aller Nachbarn des Knoten x gilt also

$$W = x \in K \mid \exists\, y \in K \rightarrow y \in N(x) \wedge y \notin S$$

Die Ausbreitung der Wave front ist ausschlaggebend für die Performance eines Pathfinding Algorithmus. Schließlich hängt sie direkt damit zusammen, wie viele Knoten der Algorithmus bearbeiten muss, bevor er terminiert.

Die erste wichtige Frage, die sich dabei stellt, ist, welche Knoten als benachbart angesehen werden. Diese Frage hat nicht nur Auswirkungen darauf, wie die Wave front sich ausbreitet, sondern kann auch Auswirkungen auf das Ergebnis haben.

Wie Emerson beschreibt, kann sich eine diamantförmige Konstellation um das Ziel herum ergeben, wenn nur die vier direkten Nachbarn und nicht alle acht Nachbarn inklusive der Diagonalen betrachtet werden [1].

Für unsere Anwendung im dreidimensionalen Raum ist die maximale Anzahl der Nachbarn aber nicht 4 und 8, sondern 6 und 26 (siehe Abbildung 9). Die richtige Wahl ist hier also kritisch.

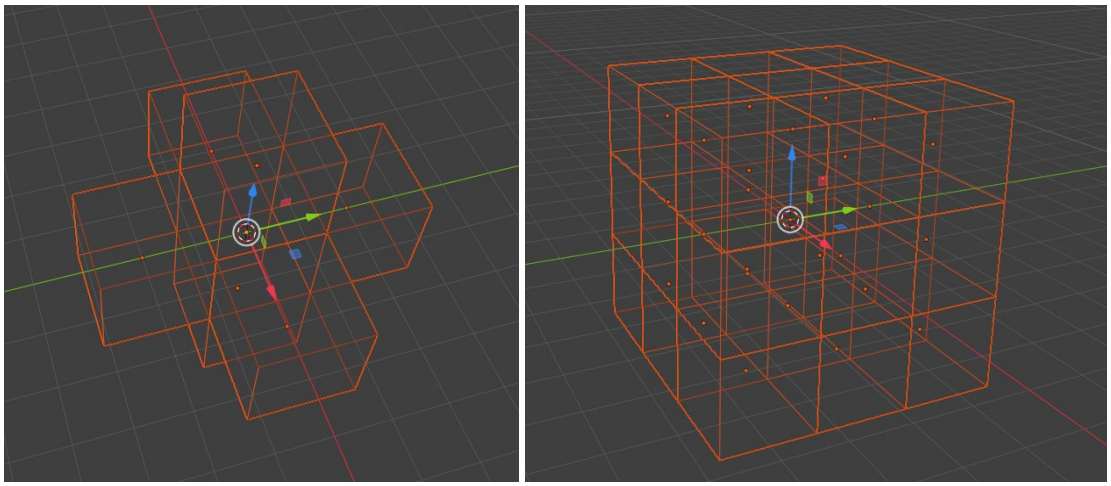


ABBILDUNG 9: EINE ZELLE MIT 6 NACHBARN (LINKS) EINE ZELLE MIT 26 NACHBARN (RECHTS) (EIGENEN ABBILDUNG)

Emerson beschreibt, dass der Algorithmus für Supreme Commander 2 nur die 4 direkten Nachbarn betrachtet. Er empfiehlt nicht alle 8 Zellen zu betrachten, begründet dies aber nicht weiter.

In dem für diese Arbeit verwendeten Netz aus uniformen Zellen, sollte alle benachbarten Zellen zu betrachten aber keine Einschränkung sein, da auf jeder dieser Zellen direkt und mit vernachlässigbarem Aufwand zugegriffen werden kann. Vielmehr sollte der Pfad näher an einer tatsächlich optimalen Lösung sein, wenn die Pfade auch über Diagonale Verbindungen geknüpft werden können.

Sollte der Graph auf einer komplexen, anstatt einer gleichmäßigen Geometrie basieren, ist es an dieser Stelle notwendig, die Ausbreitung der Wave front weiter zu optimieren. Für den Graphen in unseren Fall ist das nicht nötig.

4.2 VEKTOREN BERECHNEN

Nachdem A* erfolgreich einen Pfad gefunden hat muss nun noch für jede Zelle der entsprechende Vektor berechnet werden.

Wenn bei der Ausführung von A* alle wichtigen Daten gespeichert wurden, ist dieser Schritt sehr einfach. Dazu wird eine Liste P mit allen Zellen, die auf einem von A* berechneten Pfad liegen, benötigt.

Jede dieser Zellen z kennt seinen Nachfolger n auf diesem Pfad. Dann wird der Vektor v für jede Zelle berechnet mit:

$$v = center_z - center_n$$

Der Vektor v zeigt also der Mitter der Zelle z zu der Mitte der Zelle n. Dies ist die Richtung in die sich ein Agent der sich in der Zelle z befindet, um den Spieler zu erreichen. Daraus ergibt sich eine Kette an Vektoren, die vom Agenten zum Spieler führt.

5 PERFORMANCE

Das Folgende Kapitel dokumentiert die Performance der relevanten Algorithmen, die im Rahmen dieser Arbeit implementiert wurden. Diese wird durch Dauer der Ausführung, unter verschiedenen Voraussetzungen gemessen. Außerdem wird die Anzahl der nötigen Operationen, sowie wie viele Operationen diese benötigen, um zu terminieren. Jeder Konstellation wurde 3-mal getestet, um sicher zu stellen, dass die angegebenen Werte repräsentativ sind.

5.1 ERSTELLEN DES GRAPHEN

Tabelle 1 zeigt wie lange die Ausführung des Algorithmus zum Erstellen des Graphen gedauert hat. Erfasst wurden Ergebnisse für die gleiche Szene mit veränderter Zellgröße. Damit ändert sich dann die Anzahl der Zellen, in die die Szene unterteilt wird.

TABELLE 1

Größe der Zellen (in Unity Einheiten)	Anzahl der Zellen	Execution Time in Millisekunden
0,5	311284	106939
0,6	212167	66762
0,7	158662	44851
0,8	123590	31948
0,9	98945	22930
1	85175	19437
2	24436	3744
3	12907	1581

Abbildung 10 zeigt, wie die Dauer der Ausführung mit der Anzahl der Zellen in der Szene steigt. Man sieht das Wachstum ist annähernd linear. Dieses Verhalten war zu erwarten, es gibt keine Optimierung in diesem Algorithmus, die die Wachstumskurve anders gestalten sollten.

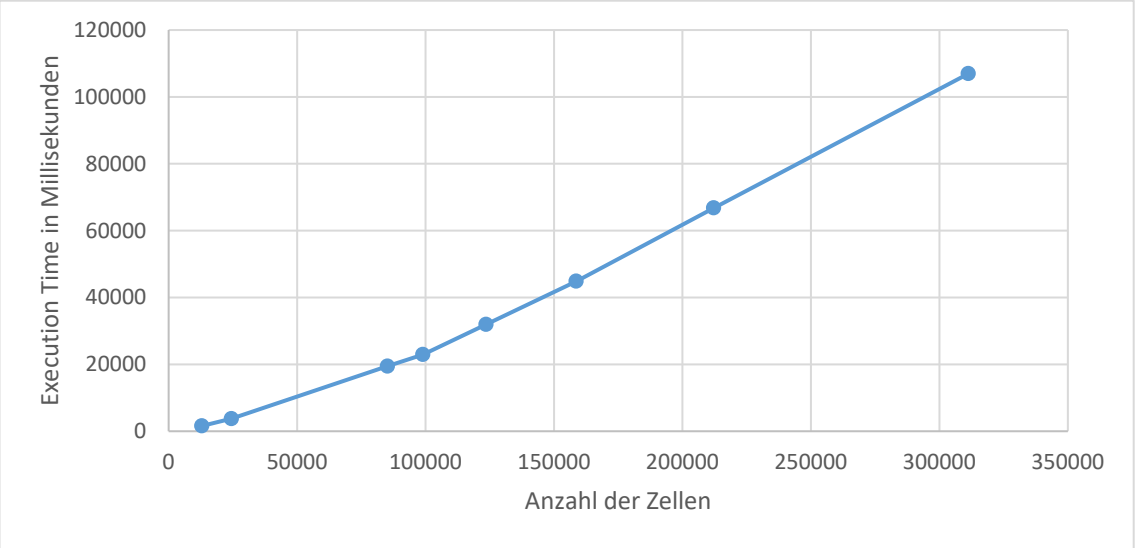


Abbildung 10 Das Wachstum bei steigender Anzahl der Zellen (Eigenen Abbildung)

Abbildung 11 zeigt, wie die Anzahl der Zellen steigt, umso kleiner die Größe der Zellen gewählt wird. Wie in Kapitel 3 erläutert wird das Wachstum asymptotisch für Wert unter 1.

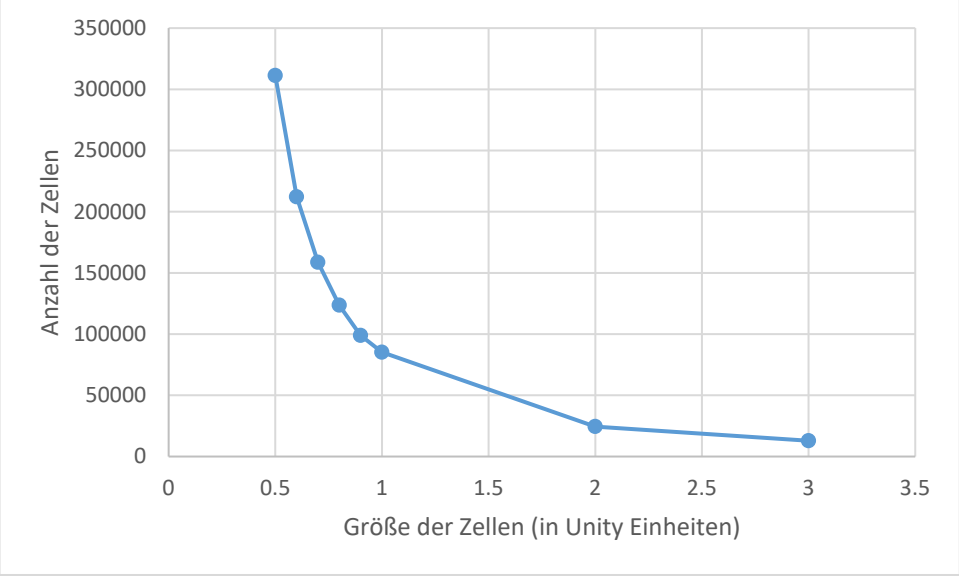


Abbildung 11 Die Anzahl der Zellen mit zunehmender Größe der Zellen (Eigenen Abbildung)

5.2 A*

5.2.1 Wachstum bei steigender Zahl der Agenten

Tabelle 2 beinhaltet die Versuchswerte für die Ausführung des A* Algorithmus für eine variable Anzahl an Agenten in der Szene. Die Position der Agenten wird dabei nicht angepasst, zwischen den Versuchen wurden lediglich mehr Agenten in der Szene platziert an den exakt gleichen Stellen. Da A* über mehrere Ticks berechnet wird, ist die angegebene Zeit die Summe der einzelnen Zeiten. Die Anzahl der Zellen pro Tick war auf ein Maximum von 700 begrenzt. Die Spalte „Summe Zellen“ gibt an, wie viele Zellen von A* berechnet wurden.

Tabelle 2

Summe Agenten	Summe Zellen	Execution Time in Millisekunden	Anzahl Ticks
1	4260	25	7
5	4374	31	7
50	4455	76	7
100	6956	127	10
250	9448	271	14
500	11830	607	21
1000	12282	887	20

Abbildung 12 zeigt, wie die Dauer der Ausführung mit steigender Zahl der Agenten steigt. Dieses Verhalten ist unerwünscht, da sich die Positionen der Agenten nicht verändern. Wie Abbildung 13 zeigt steigt aber auch die Zahl der von A* berechneten Zellen, wenn auch nicht so stark wie die Dauer der Ausführung. Das Problem scheint also bei einer nicht optimalen Verwaltung der Liste der Agenten zu liegen. Bei der nächsten Version dieses Algorithmus ist das eine Stelle, an der Optimierungen vielversprechend sind.

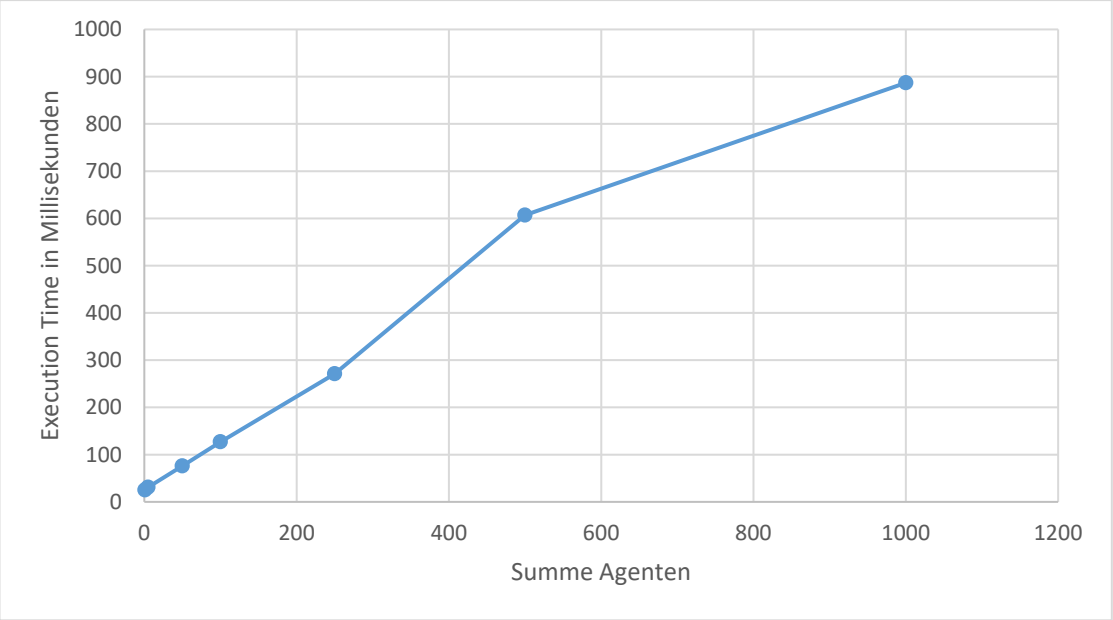


ABBILDUNG 12 WACHSTUM MIT STEIGENDER ANZAHL DER AGENTEN (EIGENEN ABBILDUNG)

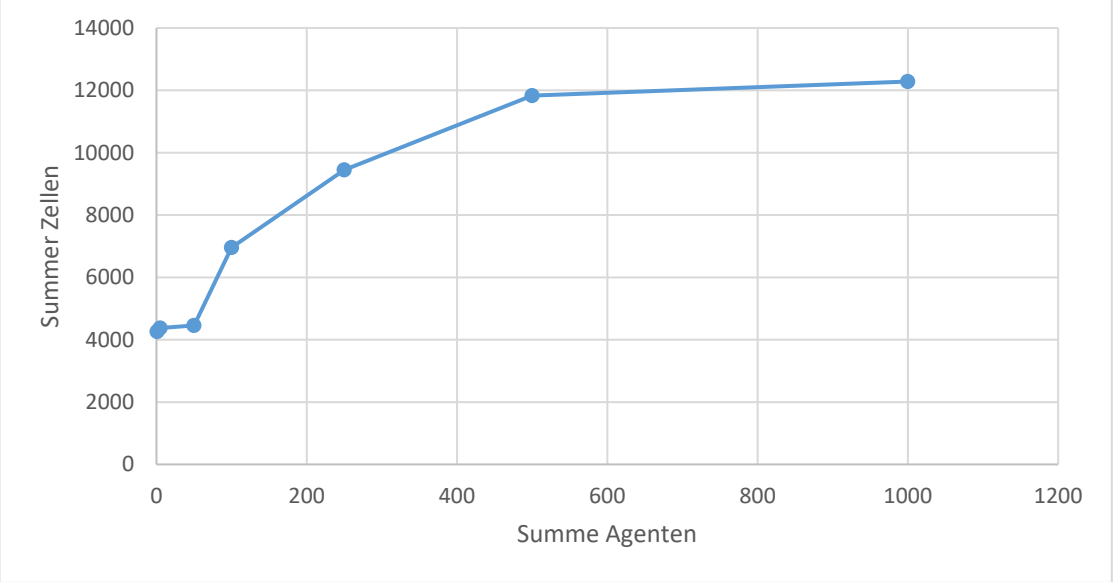


ABBILDUNG 13 STEIGENDE ZAHL DER ZELLEN BEI STEIGENDER ZAHL DER AGENTEN (EIGENEN ABBILDUNG)

5.2.2 WACHSTUM BEI STEIGENDER ANZAHL DER ZELLEN

Tabelle 3 zeigt wie lange die Ausführung von A* dauert bei steigender Zahl der zu berechnenden Zellen. Für diese Tests wurde nicht die Anzahl der Agenten erhöht. Stattdessen wurden die 10 in der Szene vorhandenen Agenten immer weiter verteilt, so dass A* immer größere Teile des Graphen lösen muss. Da A* über mehrere Ticks berechnet wird, ist die angegebene Zeit die Summe der einzelnen Zeiten. Die Anzahl der Zellen pro Tick war auf ein Maximum von 700 begrenzt.

TABELLE 3

Summe Zellen	Execution Time in Millisekunden	Anzahl Ticks
4396	31	7
11775	121	17
12392	135	18
16071	115	23
19563	144	28
25678	215	37
28561	238	41

Abbildung 14 zeigt wie die Dauer der Ausführung des A* für eine steigende Zahl der zu berechnenden Zellen, ebenfalls zunimmt. Das Wachstum ist annähernd linear. Für 100 Zellen benötigt der Algorithmus im Schnitt 0,8 Millisekunden. Dieses Verhalten ist positiv zu bewerten, da selbst bei einer suboptimalen Verteilung der Agenten, die Laufzeit keine Sprünge nach oben macht.

Was die Zahlen nicht zeigen, ist das der A* Algorithmus auf dem Weg zu den verstreuten Agenten immer sehr effiziente Wege gefunden hat und wenige Zellen besucht hat, die nicht auf dem optimalen Pfad liegen. Um die Summe der Zellen zu erhöhen, mussten die Agenten in alle Ecken der Szene verteilt werden.

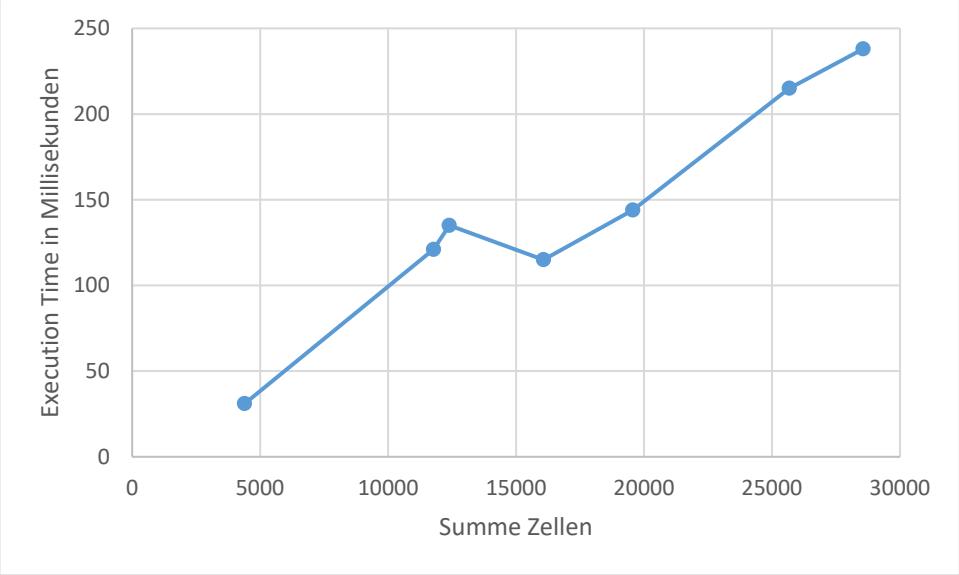


ABBILDUNG 14 DIE AUSFÜHRUNGSZEIT STEIGT MIT DER SUMMER DER ZELLEN (EIGENEN ABBILDUNG)

6 OPTIMIERUNGEN

Es wurde bis hierhin bereits beschrieben, wie ein grundlegender Algorithmus funktioniert, der mit effizienten Berechnungen, die Navigation durch eine komplexe 3D Szene für eine hohe Anzahl an Agenten ermöglicht. Das Folgende Kapitel wird auf weitere Ansätze eingehen, die weiter die Performance dieses Algorithmus optimieren können.

Basierend auf dem von Emerson beschrieben Ansatz der Portale, wird eine Methode erläutert, um die Anzahl der Zellen, die der A* Algorithmus durchlaufen muss, zu minimieren. Auch ähnlich dem Konzept von Emerson wird auf variable Zellgrößen eingegangen mit dem gleichen Ziel, die Anzahl der zu bearbeitenden Zellen zu reduzieren.

Weiter wird noch die Ausführung des A* über mehrere Ticks hinweg besprochen, um die Performance zu verbessern. Zuletzt werden noch Ansätze der „Level of Detail Skalierung“ für die Navigation erläutert.

6.1 PORTALE

Emerson beschreibt in seinem Artikel, dass das 2-dimensionale Spielfeld von Supreme Commander 2 in gleich große Sektoren unterteilt wird. Alle diese Sektoren haben an den Schnittstellen zu benachbarten Sektoren sogenannte Portale.

Diese Portale enden an einer nicht begehbaren Zelle oder an der Grenze zu einem anderen Sektor [1]. In Supreme Commander 2 wird dann der A* Algorithmus auf einem Graphen ausgeführt, dessen Knoten die Portale sind [1].

Der A* wird im Modus „merging“ ausgeführt. Er terminiert, wenn er auf einen schon bestehenden Pfad trifft. Diese Portale werden dafür benutzt, die Anzahl der Zellen, die auf einmal berechnet werden müssen, zu begrenzen. Es wird immer nur das Flow-Field bis zum nächsten bzw. übernächsten Portal befüllt. Somit ist die Rechenlast pro Tick immer in einem geringen Rahmen.

Die Portale werden als eine Art Level of Detail Skalierung des Pathfinding eingesetzt. Der gesamte Pfad wird nur grob über den Graphen der Portal-Knoten berechnet. Der exakte Pfad wird immer nur für einen oder zwei Sektoren berechnet, bis das gesuchte Portal erreicht ist.

Dieser Ansatz lässt sich nicht direkt auf ein Spiel in einer komplexen 3D Szene übertragen. Diese Portale entlang regelmäßig generierter Sektorgrenzen zu definieren, ist ein erhebliches Problem. Der einfache Ansatz, die Portale analog zum 2D Beispiel zu generieren, also diese durch Sektor-Grenzen und Wände zu beschränken, stößt schnell auf Probleme.

Wenn sich innerhalb eines Sektors auf mehreren y-Leveln Zellen befinden, beispielsweise mehrere Etagen eines Hauses, macht es keinen Sinn diese Zellen zu einem Portal

zusammenzufassen. Portale können aber auch nicht einfach durch unterschiedliche y-Level voneinander getrennt werden, schließlich kann eine Sektor-Grenze auch entlang einer Schräge verlaufen.

Sicher kann auch dafür eine Lösung gefunden werden, beispielsweise Portale durch eine leere Zelle in Y-Richtung zu trennen. In dieser Arbeit wurde aber ein anderer Ansatz verfolgt, nämlich die Portale manuell an engen Stellen der Szene (bspw. Türen, Tunnel, etc.) zu setzen und diese zur Minimierung der Kalkulationen einzusetzen.

Ein weiterer Ansatz um wie die Berechnung auf mehrere Ticks verteilt werden kann, wird im Kapitel 6.3 „Dynamische Ticks“ erläutert.

Bei der Erstellung der Szene werden Blöcke genau dort gesetzt, wo später ein Portal entstehen soll. Alle Zellen, die dieser Block berührt, werden beim Laden der Szene als ein Portal markiert. Anders als bei Emerson werden nun diese Portale aber nicht als Knoten für den A* Algorithmus verwendet. Die Wirkung der Portale während der Laufzeit sieht wie folgt aus:

1. Bei der ersten Ausführung von A* haben die Portale noch keine Auswirkung.
   Es wird lediglich vermerkt, welches Portal auf einem Pfad zuerst erreicht wurde. Auf dem Pfad vom Spieler zum Agenten wird für jeden Agenten gespeichert, hinter welchem Portal dieser Agent liegt (siehe Abbildung 15).

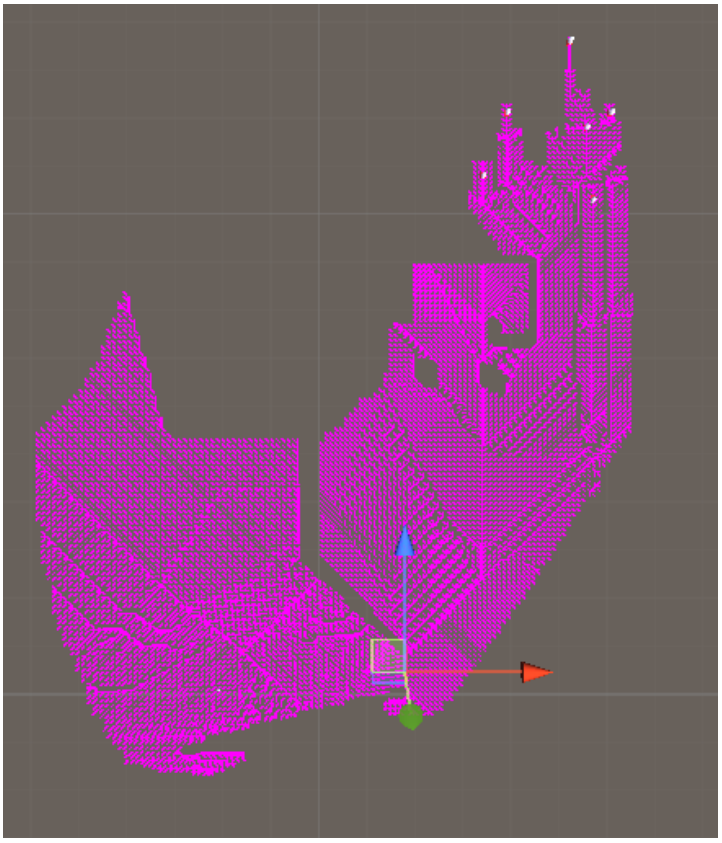


ABBILDUNG 15: DAS ERGEBNIS BEIM ERSTEN DURCHLAUF (EIGENE ABBILDUNG)

2. Bei der nächsten Ausführung können dann, wenn das entsprechende Portal erreicht wurde, alle jene Agenten, die hinter diesem Portal liegen, als erreicht betrachtet werden. Es wird also nur die Seite des Portals neu berechnet, auf der sich der Spieler befindet und bewegt. Der Teil des Pfades, der hinter dem Portal liegt, bleibt unverändert. (siehe Abbildung 16)

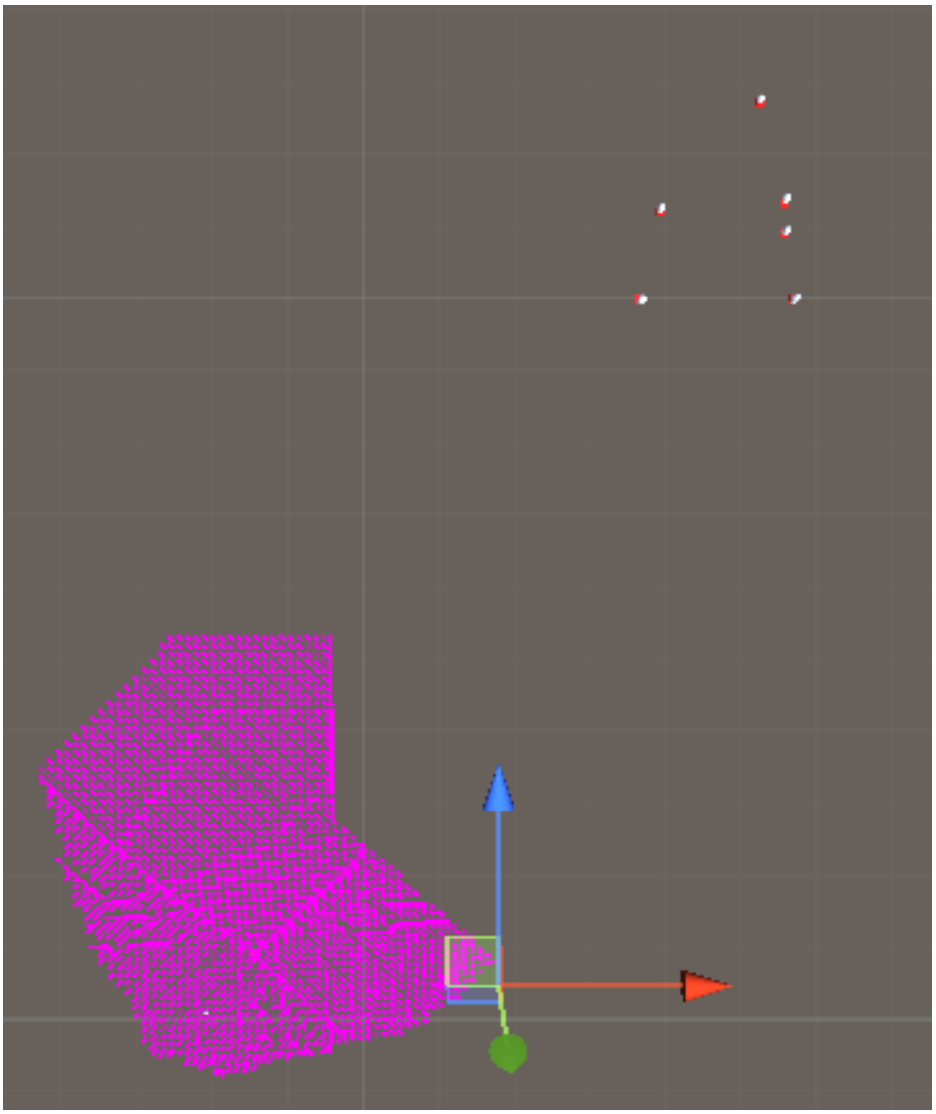


ABBILDUNG 16: ERGEBNIS DES ZWEITEN DURCHLAUFS. ZU SEHEN SIND NUR JENE ZELLEN, FÜR DIE EIN NEUER VEKTOR BERECHNET WURDE. (EIGENE ABBILDUNG)

Bei diesem Vorgehen ist die Ersparnis stark davon abhängig, wo sich der Spieler in Relation zum Portal befindet. Optimal ist eine möglich kurze Distanz zum Portal, beispielsweise wenn sich der Spieler in einem Raum bewegt und die Tür das Portal ist. Der Nachteil ist allerdings, wenn die Situation umgekehrt ist, der Agent sich also nahe am Portal und der Spieler weit davon weg befindet. In diesem Fall sind die Ersparnisse zu vernachlässigen.

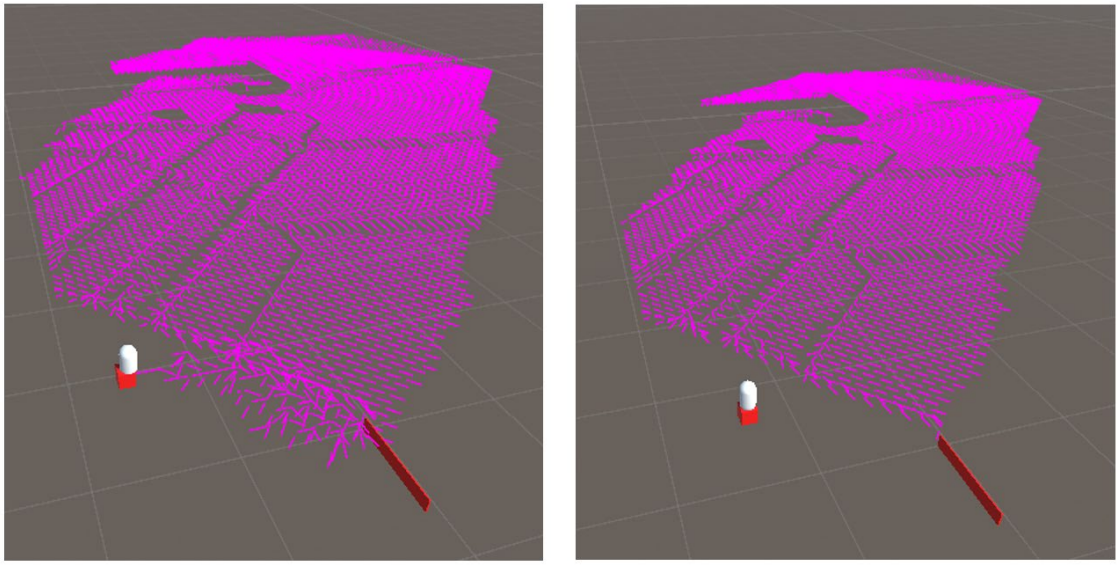


ABBILDUNG 17: DER ERSTE DURCHGANG (LINKS) DER ZWEITE DURCHGANG (RECHTS) DAS PORTAL IST IN ROT ZU SEHEN. (EIGENE ABBILDUNG)

Außerdem gibt es für Szenarios, in denen kein Portal auf dem Weg liegt, beispielsweise wenn die Szene eine Wiese oder Waldstück ist, nichts, was die Ausführung des A* Algorithmus nach oben begrenzt. In vielen Fällen muss A* eine hohe Anzahl an Knoten durchwandern (siehe A* Trap)[G], oder im Worst-Case kann gar kein Weg zum Agenten gefunden werden. In diesem Fall würde dann A* den gesamten Graphen lösen, also zu allen erreichbaren Zellen einen Pfad berechnen.

Diesem Problem kann man aus dem Weg gehen, indem der Graph zusammenhängend ist, also zwischen allen Zellen $x, y \in G$ mit $x \neq y$ ein Pfad existiert.

Diese Version der „Portale“ ist also flexibel und zuverlässig einsetzbar für eine komplexe 3D Szene. Der Aufwand bei der Erstellung ist gering und die potentielle Ersparnis erheblich. Sie stellt aber nicht sicher, dass die Berechnungen pro Tick sich in einem Rahmen halten, der sich nicht negativ auf die Framerate des Spiels auswirkt. Dafür bedarf es weitere Optimierung.

6.2 DYNAMISCHE ZELLGRÖßE

Ein weiterer Ansatz, der an die Idee anknüpft, die Navigation auf einem Graphen mit einer reduzierten Anzahl an Knoten durchzuführen, liegt darin, die bestehenden Zellen zusammenzufassen. Gemeint ist, zusammenhängende Cluster an Zellen ausfindig zu machen, in denen alle Zelle begehbar sind.

Nehmen wir beispielsweise eine Wiese, auf der sich keine Hindernisse befinden können. Hier können Zellen zu einer größeren Zelle zusammengefasst werden, um die Anzahl der Knoten des Graphen zu verringern. Dies würde die Rechenzeit für den A* Algorithmus verkürzen. Gleichzeitig würden aber auch die Probleme wieder auftreten, die mitigiert werden sollten, als

die richtige Größe für die Zellen gewählt wurde. Die Portale, also einzelne Punkte als Knoten zu wählen, macht hier mehr Sinn, als Knoten zusammenzufassen, auch wenn es das gleiche Ziel hat. Schließlich repräsentiert das Portal immer noch einen relativ kleinen Bereich zu dem navigiert wird, im Gegensatz zu einer Zelle mit erheblicher Ausdehnung, die im Graphen aber wiederum durch einen einzogen Punkt repräsentiert werden muss. Außerdem gibt es einen Konflikt mit dem Prinzip des direkten Zugriffs auf Zellen über die Koordinaten des Agenten, wenn Zellen nicht mehr eine uniforme Größe haben, der sich zwar lösen lässt aber den Code weiter kompliziert.

Der Ansatz, die Größe der Zellen anzupassen, um die Zahl der Knoten zu reduzieren, passt also nicht gut zusammen mit dem in dieser Arbeit verfolgten Ansatz.

6.3 DYNAMISCHE TICKS

Für die Performance eines Videospiels ist es wichtig, dass keiner der einzelnen Ticks der Game-Engine eine gewisse Dauer bei der Berechnung überschreitet, da sich dies in Form von Lag-Spikes negativ auf das Spielerlebnis auswirkt.

Ein vollständig berechneter A* Algorithmus, in jedem Tick, in dem sich der Spieler bewegt, würde dafür sorgen, dass die Ticks regelmäßig diese Dauer überschreiten. Es ist also wichtig, zwei Aspekte gleichmäßig auf die Ticks der Game-Engine zu verteilen:

1. Wie häufig wird eine Neuberechnung der Navigation ausgeführt?
   Theoretisch könnte dies in jedem Tick geschehen, in dem sich der Spieler bewegt hat. Dies ist allerdings nicht nötig, da sich wenig bis gar nichts ändern wird, falls die Distanz zum Startpunkt der letzten Berechnung gering ist.
2. Über welche Anzahl an Ticks wird eine Ausführung von A* aufgeteilt, um die Last gleichmäßig zu verteilen, damit keiner der Ticks einen spürbaren Lag erfährt.

Für den ersten Punkt gibt es eine naheliegende Lösung. Beim Aufrufen des A* wird die Position des Spielers gespeichert. Die nächste Ausführung wird dann erst ausgelöst, wenn eine festgelegte Distanz zwischen dem gespeicherten und dem aktuellen Standort des Spielers besteht. Die naheliegende Wahl für diese Distanz ist die Zellengröße, damit eine Neuberechnung dann ausgelöst wird, wenn sich der Spieler in einer neuen Zelle befindet. Unter bestimmten Umständen, beispielsweise wenn die Agenten noch nicht im Nahbereich des Spielers sind, kann die Distanz auch größer gewählt werden (2-3 Zellgrößen) um A* Ausführungen seltener zu machen.

Für den zweiten Teil gibt es auch eine intuitive Lösung, nämlich die Anzahl der Zellen, die von A* pro Tick bearbeitet werden, auf eine feste Anzahl zu begrenzen. Das ist tatsächlich auch eine Lösung, die für die Testszene dieser Arbeit gut funktioniert hat. In diesem Fall wurde die komplette Berechnung auf zwei Ticks aufgeteilt. Diese feste Begrenzung war hier also

ausreichend. Games werden allerdings auf hunderten verschiedenen Systemen, mit unterschiedlicher Leistungsfähigkeit ausgeführt.

Am besten ist ein dynamischer Ansatz, der die maximale Anzahl der Zellen anpasst, um die Dauer eines Ticks innerhalb einer oberen Schranke zu halten. Diese Schranke ist abhängig von der angestrebten Framerate. Die maximale akzeptable Dauer für einen Tick in Sekunden ist $\frac{1}{Framerate}$. Während der Ausführung kann die Dauer der Ticks erfasst werden und falls es zu Überschreitungen dieser Schranke kommt, die maximale Anzahl der Zellen reduziert werden. So kann eine stabile Framerate garantiert werden.

Schließlich muss noch der Fall behandelt werden, was passiert, wenn eine neue Berechnung von A* ausgelöst wird, die vorherige Ausführung aber noch nicht beendet wurde. Sollte dies während der regulären Bewegung des Spielers geschehen, ist das ein Hinweis darauf, dass die Distanz für die Neuberechnung zu niedrig gewählt ist. Es kann aber trotzdem passieren, dass zwei Berechnungen sich überschneiden. In diesem Fall sollte die neuere Berechnung ausgeführt werden und die ältere verworfen werden.

6.4 NAHBEREICH LINE OF SIGHT NAVIGATION

In Kapitel 3 „Erstellen des Graphen“ wurde bereits ein Ansatz besprochen, die Anzahl der Knoten zu skalieren, um Teile der Navigation, die nicht exakt sein müssen, effizienter zu gestalten. Es gibt noch einen weiteren Ansatz, unterschiedliche Level an Genauigkeit für die Navigation zu nutzen.

Emerson beschreibt, dass in Supreme Commander 2, für den Bereich, in dem eine direkte Sichtlinie zum Ziel besteht, die Navigation nach einem anderen Verfahren berechnet wird. Emerson tut dies, weil es im 2D Raum eine sehr effiziente Operation ist, direkte Linien von einem Punkt aus zu berechnen [1].

Außerdem löst Emerson damit das Problem, dass sich manchmal „diamond-shaped flow directions“ [1] ergeben. Die Agenten würden sich dann auf zwei Zellen neben dem Ziel sammeln. Dieses Problem tritt allerdings bei dem in dieser Arbeit gewählten Ansatz nicht auf, da der A* Algorithmus bei seiner Ausbreitung nicht nur die 4 (bzw. in 3D 6) direkt benachbarten Zellen betrachtet, sondern alle Zellen, inklusive der diagonal benachbarten Zellen (8 in 2D, 24 in 3D).

Die Motivation, die im Falle von Supreme Commander ausschlaggebend war, ist hier also nicht relevant. Der Ansatz, für den Nahbereich des Spielers eine andere Navigation zu verwenden als für den Rest des Pfades, ist aber trotzdem einen zweiten Blick wert.

Befinden sich die Agenten in der Nähe des Spielers, wird es wichtig, dass die Agenten ihren Weg exakt zum Spieler finden und nicht zum Mittelpunkt der Zelle, in der sich dieser befindet. Dabei sollte es auch keine Verzögerung geben, ausgelöst durch unregelmäßige Updates.

Solche Fehler fallen dem Spieler direkt auf und haben einen direkten Einfluss auf das Spielerlebnis. Deshalb kann es, je nach Spiel, sinnvoll sein, für den Bereich in der unmittelbaren Umgebung des Spielers eine andere Zielfindung zu nutzen.

Damit ein Agent die richtige Zielführung nutzt, kann bei der Ausführung des A* bis zu einer festgelegten Distanz ein Flag gesetzt werden, das Nahbereich-Zielführung repräsentiert. Je nach Anwendung kann auch ein Line-Of-Sight Test mit in die Entscheidung einfließen, welche Navigation verwendet wird. Für die Zwecke dieser Arbeit war eine feste Distanz ausreichend. Die Alternative Zielführung kann dann den eingebauten Navigationsalgorithmus der jeweiligen Engine verwenden.

Aus zwei Gründen führt dieses Vorgehen zu keinen Performance Problemen

1. Die Anzahl der Agenten, die sich gleichzeitig in der Nähe des Spielers befinden, ist in einer realen Spielsituation immer nur ein kleiner Teil aller Agenten.
2. Wenn die Distanz zum Ziel gering ist, sind auch nicht optimierte Pathfinding-Algorithmen schnell genug.

Für einen kleinen Bereich um den Spieler herum einen anderen Ansatz zu wählen, kann also für das Spielerlebnis von Vorteil sein.

6.5 MERGING

In dem Kapitel zum A* Algorithmus wurde bereits erläutert, wie durch Merging die Laufzeit von A* dramatisch verringert werden kann. Es lohnt sich zusätzlich Merging unter einem anderen Gesichtspunkt zu betrachten, nämlich des Merging zwischen zwei Ausführungen des A* Algorithmus.

Betrachtet man, wieviel der Zellen sich pro Ausführung ändern, sieht man einige Veränderung im Bereich um den Spieler und wenig bis gar keine Veränderung für den Rest der Pfade (siehe Abbildung 18).

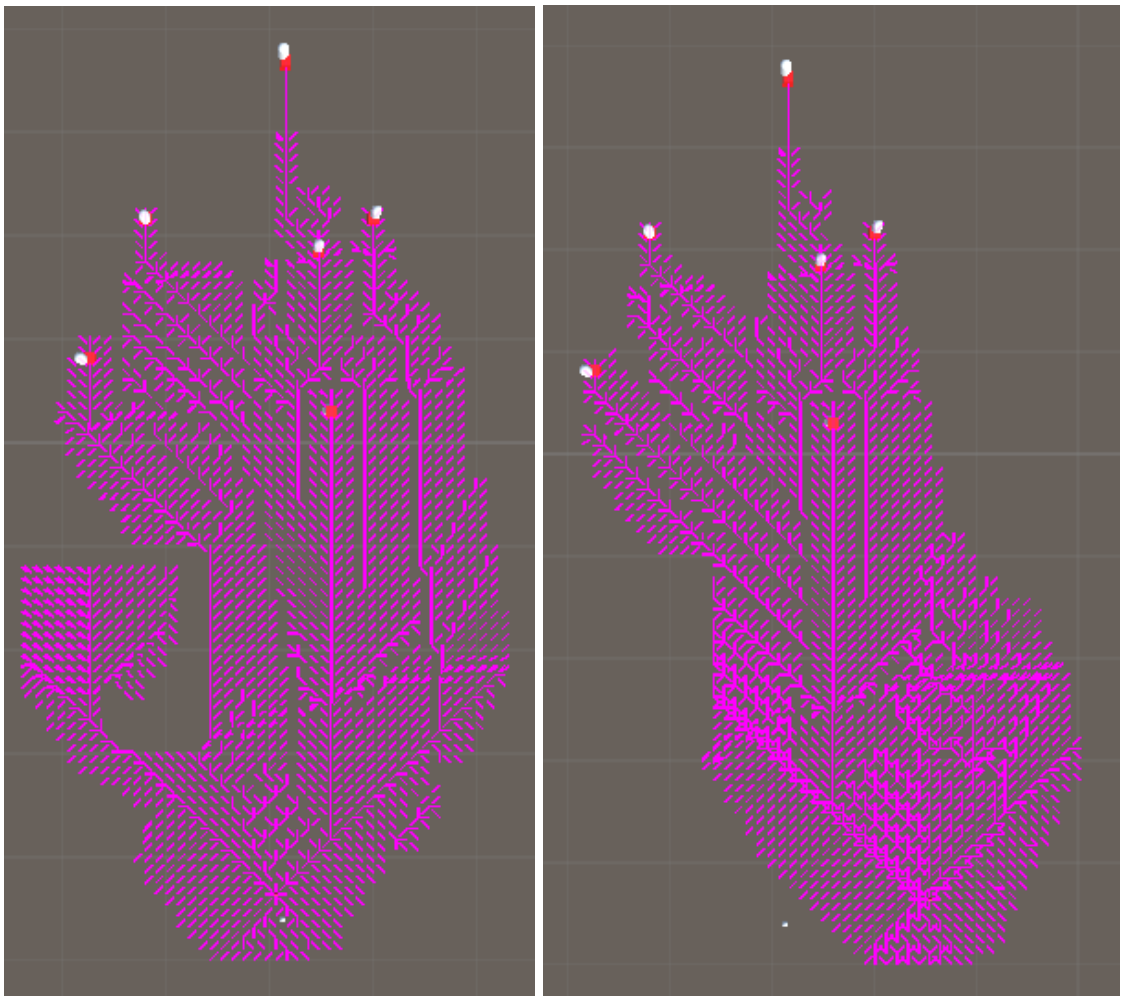


ABBILDUNG 18: DER SPIELER HAT SICH UNGEFÄHR 10 ZELLEN ZWISCHEN DEN BEIDEN AUSFÜHRUNGEN VON A* BEWEGT (EIGENE ABBILDUNG)

Die Idee liegt also nahe, Merging auch zwischen der aktuellen und der vorherigen Ausführung von A* zu nutzen. Gemeint ist, dass ein gewisser Bereich um den Spieler neu kalkuliert wird und ab einer festgelegten Distanz terminiert der Algorithmus, wenn er auf eine Zelle trifft, die schon bei der letzten Ausführung erreicht wurde.

Der Effekt dieses Vorgehens ist vergleichbar mit dem Fall, das sich der Spieler direkt hinter einem Portal aufhält. Dabei wird das Ergebnis von A* nicht mehr optimal sein. A* kann also nicht dauerhaft in diesem Merging-Modus betrieben werden. Regelmäßig muss A* vollständig ausgeführt werden, um sicher zu stellen, dass der Pfad nicht zu weit vom korrekten Pfad abweicht. Zwei mögliche Ansätze könne dieses Ergebnis herbeiführen:

1. Eine feste Abfolge wird festgelegt
   (bspw. zwei merging- gefolgt von einer nicht-merging-Ausführung)
2. Nachdem der Spieler sich eine gewisse Distanz zurückgelegt hat, wird wieder ein vollständiger A* berechnet.

Grundsätzlich ist dieses Vorgehen keine optimale Lösung und eher als Möglichkeit anzusehen, wenn die Anzahl der Agenten besonders hoch ist und alle verfügbaren Optimierungen verwendet werden müssen.

7 GAME DESIGN

Der in dieser Arbeit beschriebene Algorithmus soll eine effiziente Lösung für eine hohe Anzahl an Agenten beschreiben. Bis jetzt wurden Game-Design-Aspekte dieser Anwendung gar nicht behandelt.

Dieses Kapitel befasst sich mit Ansätzen, die sich auf das Spielerlebnis auswirken, vor allem mit Aspekten des Game Designs, die im starken Zusammenhang mit dem Pathfinding stehen. Dabei wird ein Ansatz erläutert, wie bei einer großen Gruppe Agenten verhindert wird, dass alle Agenten den exakt gleichen Weg wählen, mit dem Ziel, ein realistischeres Verhalten zu erzielen.

Zusätzlich wird anhand des Spiels „Days Gone“ erläutert, wie realistisches Verhalten in Bezug auf das High-Level-Verhalten der Agenten, erzielt werden kann.

Zuletzt wird noch auf Animationen und Modelle für eine hohe Anzahl Agenten eingegangen, die gleichzeitig performant und dabei nicht uniform wirken.

7.1 ALTERNATIVE PFADE

Denkt man daran, wie große Horden an Verfolgern in Filmen dargestellt werden, hat man definitiv nicht vor Augen, dass sich die Zombies in einer sehr engen Schlange auf dem Optimalen Weg zum Helden bewegen. Manche der Zombies würden sich nach alternativen Pfaden umsehen, entweder um den Weg abzuscheiden oder weil das Gedränge zu eng wird, um sinnvoll voranzukommen.

Navigieren die Agenten aber auf dem optimalen Pfad, wird genau das passieren, vorausgesetzt die Agenten sind von ungefähr der gleichen Position aus gestartet (siehe Abbildung 19).

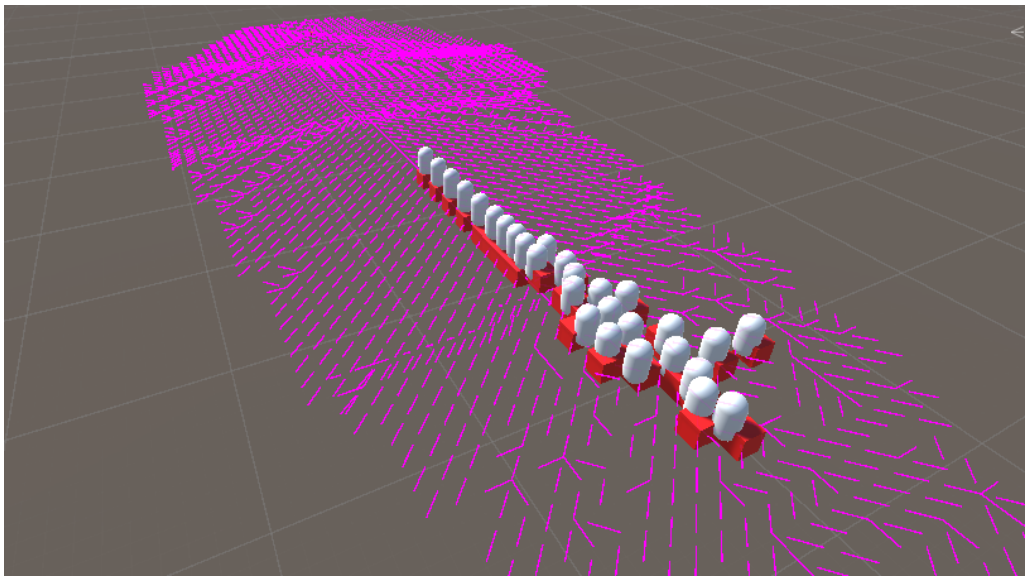


ABBILDUNG 19 DIE AGENTEN REIHEN SICH AUF DEM WEG ZUM ZIEL EIN. (EIGENE ABBILDUNG)

Und genau das ist durchaus ein realistisches Gameplay Szenario: Der Spieler trifft auf eine große Anzahl Zombies, diese sehen ihn und laufen von ungefähr der gleichen Position gleichzeitig zum Spieler. Auch die Entwicklern von „Days Gone“ beschreiben, das sie dieses Problem während der Entwicklung lösen mussten [7].

Mögliche Lösungen sind, dass die Agenten entweder von unterschiedlichen Positionen starten, zu unterschiedlichen Zeitpunkten oder beides. Dies ist aber nicht immer möglich.

Ein weiterer Ansatz ist, einen Teil der Gruppe von Agenten auf einen alternativen Pfad zu schicken. Stellt sich die Frage wie A* für die gleichen Inputs (Ziel, Start und Graph) einen alternativen Pfad liefern kann. Einer der Inputs muss verändert werden, damit sich ein anderer Pfad ergibt. Start und Zielpunkt sind gegeben, also muss der Graph modifiziert werden. Wenn wir Knoten aus dem Graphen entfernen, die auf dem Optimalen Pfad des unveränderten Graphen liegen, erhalten wir einen neuen Pfad, der nicht mehr optimal ist, aber von diesem abweicht.

Wie bestimmen wir, welche Knoten blockiert, also aus dem alternativen Graphen gestrichen werden sollen?

Die Knoten müssen auf dem optimalen Pfad liegen, sonst würde die neue Berechnung zu dem gleichen Ergebnis kommen, wie die erste Berechnung. Geht man den optimalen Pfad von dem Spieler zu der Agentengruppe und wählt nach einer festgelegten Distanz eine Zelle, kann man diese blockieren bzw. aus dem Graphen entfernen. Eine Zelle allein wird allerdings keine große Veränderung des Pfads bewirken, also muss ein Bereich um diese Zelle blockiert werden.

Das Ergebnis kann dann wie in Abbildung 20 aussehen.

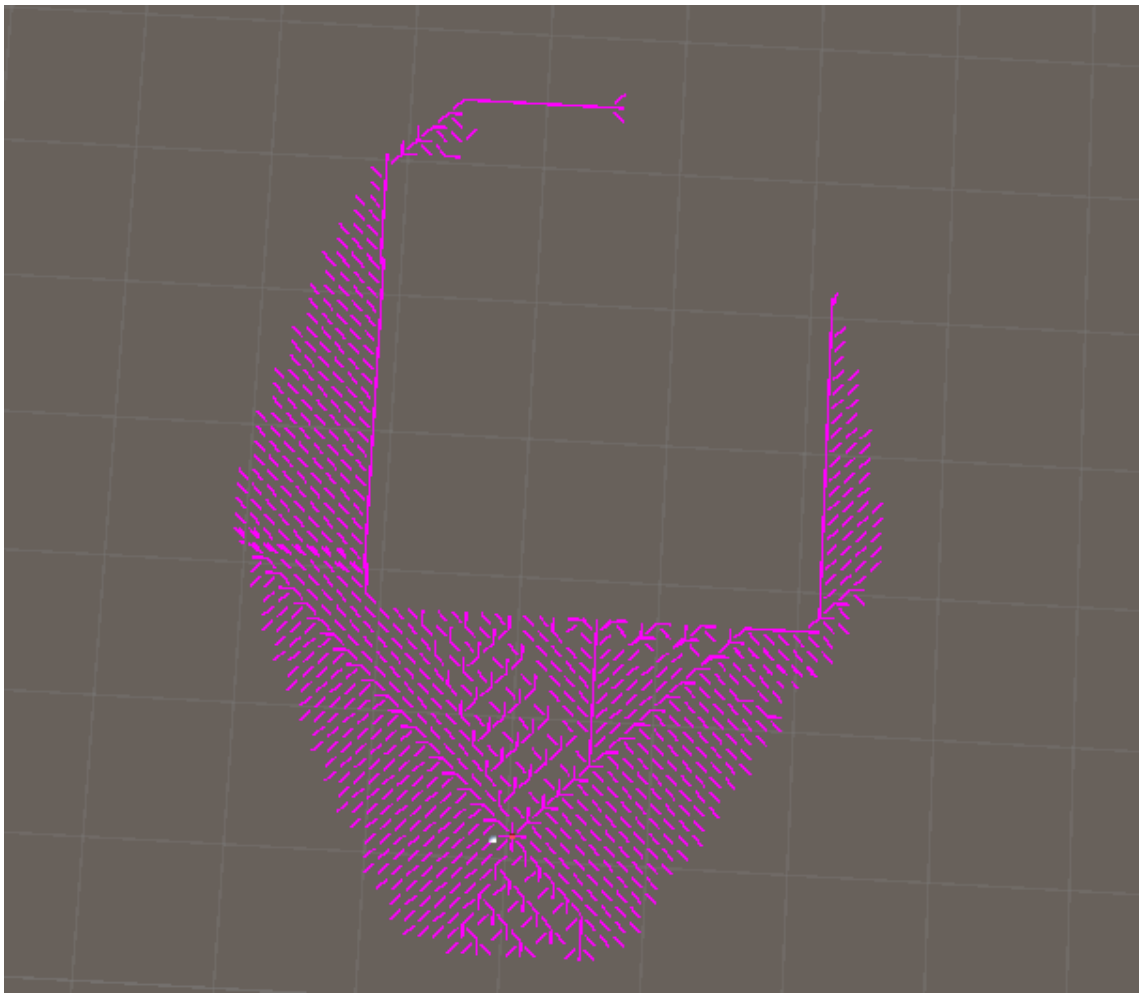


ABBILDUNG 20: DER BLOCKIERTE BEREICH IST ZU SEHEN (EIGENE ABBILDUNG)

Der Algorithmus geht dazu wie folgt vor:

1. Berechnen des A* Algorithmus + integrieren für die gesamte Gruppe ohne Modifikation des Graphen.
2. Finden der Zelle x auf dem optimalen Pfad zu einem der Agenten der Gruppe, die exakt D Zellen vom Spieler auf diesem Pfad entfernt liegt.
3. Markieren der Zelle x und aller Zellen in einem Radius R um die Zelle x, so dass A* diese als bereits erreicht ansieht. Für dieses Flag und für die kommenden Berechnungen muss die Möglichkeit bestehen, dass jede Zelle einen Alternativen Wert (sowohl A* Ergebnis als auch Vektor) speichern kann.
4. Ausführen von A* auf dem modifizierten Graphen. Als Startpunkt wird die Position des Spielers gesetzt. Als Ziel wird jene Zelle gesetzt, die auf dem optimalen Pfad liegt und exakt $D + R + 1$ Zellen auf diesem Pfad vom Spieler entfernt liegt.
5. Integrieren dieser alternativen Werte und speichern der Vektoren ohne die Werte aus Schritt 1 zu verwerfen.

Danach wird jener Prozentteil der Agenten markiert, der diesen Alternativen Pfad verwenden soll. Diese werden im Folgenden „alt-Agenten“ genannt. Der alternative Pfad führt allerdings nicht bis zu den alt-Agenten selber, sondern nur bis zu der Zelle die direkt hinter dem blockierten Bereich liegt, der von den alt-Agenten gemieden werden soll. Bis die alt-Agenten diesen Punkt erreichen, navigieren sie weiterhin über das Ergebnis, das auf dem nicht modifizierten Graphen basiert. Erreichen die alt-Agenten diesen Punkt, verwenden sie die Vektoren der zweiten Berechnung.

Dieses Vorgehen ist im allgemeinen Fall nicht sehr rechenintensiv, weil der Pfad nur zu einem Punkt mit festgelegter Distanz berechnet werden muss. Allerdings lässt sich nicht vorhersagen, wie lang der alternative Pfad ausfallen wird. Auf einem offenen Feld wird das Ergebnis immer ähnlich aussehen (siehe Abbildung 21). In anderen Situationen, besonders wenn die blockierte Stelle mit einer Engstelle der Szene zusammenfällt, kann der Weg um diese Blockade deutlich länger sein als der ursprüngliche Pfad (siehe Abbildung 22).

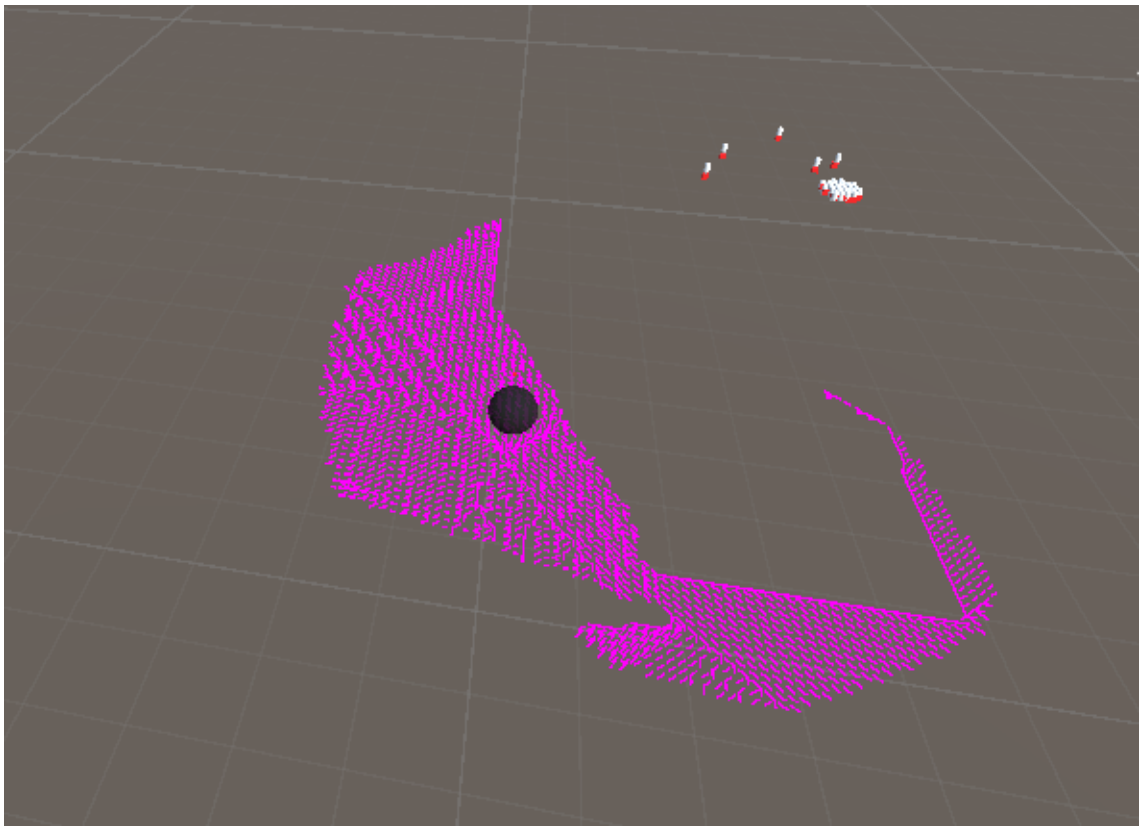


ABBILDUNG 21 ANWENDUNG AUF EINER OFFENEN FLÄCHE. SPIELER IN SCHWARZ DARGESTELLT. (EIGENEN ABBILDUNG)

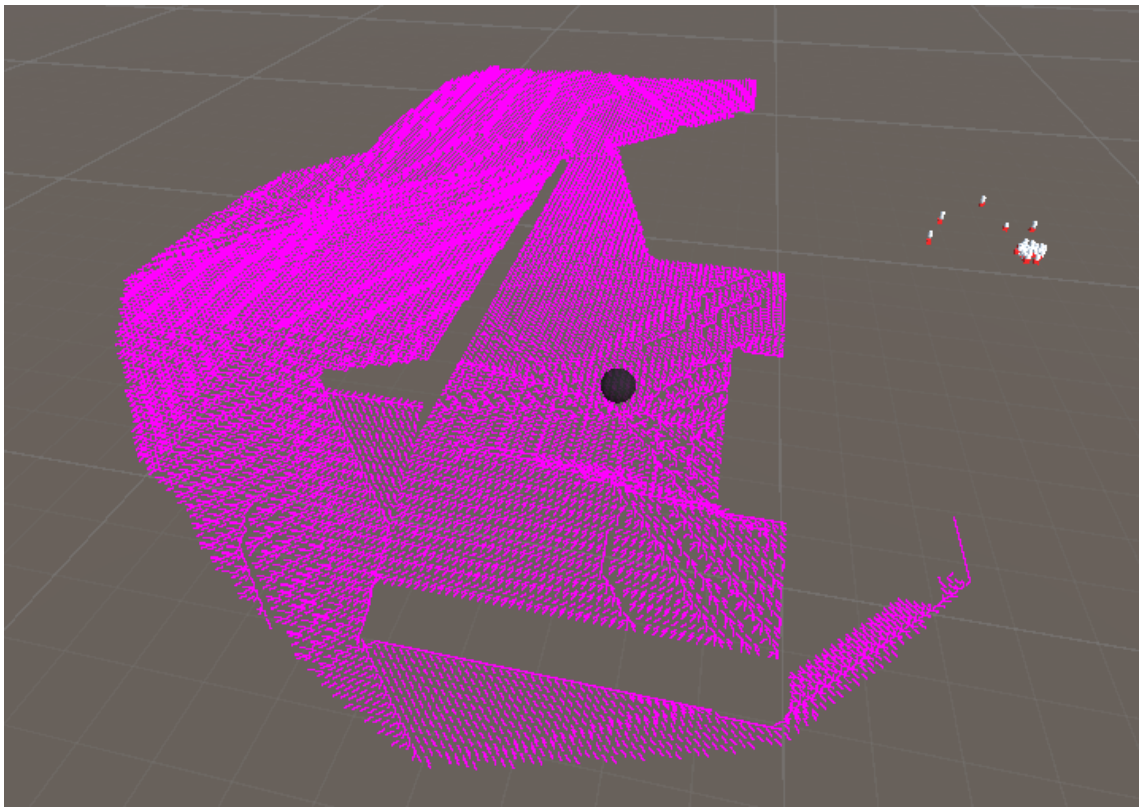


Abbildung 22: Die Blockade fällt auf eine Engstelle. Der Spieler ist in Schwarz zu sehen. (Eigene Abbildung)

Das bedeutet nicht nur erheblichen Rechenaufwand, sondern auch ein unerwünschtes Verhalten der Agenten. Wenn ein alt-Agent ohne für den Spieler ersichtlichen Grund plötzlich einen sehr weiten Umweg wählt, wird dies nicht gerade die Illusion einer intelligenten KI fördern, sondern sich eher negativ auf die Immersion auswirken. Dieses Vorgehen ist also nur für Szenen mit vielen offenen Flächen zu empfehlen. Für Szenen mit vielen Engstellen (bspw. Türen) ist das Ergebnis unberechenbar und kann sogar dazu führen, dass gar kein Weg mehr gefunden werden kann.

Dieses Vorgehen erhöht auch die Komplexität des Algorithmus. Nicht nur muss mehr als nur ein Wert pro Zelle gleichzeitig gespeichert werden. Es müssen auch mehrere Ausführungen von A*, die aufeinander aufbauen, verwaltet werden und möglicherweise über mehrere Ticks in Folge ausgeführt werden.

Grundsätzlich ist es empfehlenswert, diese Problematik nicht nur durch einen Algorithmus zu lösen, sondern bei der Entwicklung des Spiels diesen Aspekt mitzudenken. Wenn die Agenten beispielsweise zum Beginn des Levels gut verteilt sind und zu versetzten Zeitpunkten alarmiert werden, kann es passieren, dass eine algorithmische Lösung gar nicht nötig ist.

7.2 VERHALTEN DER AGENTEN

Ein Aspekt, der sowohl das Game Design als auch die Navigation betrifft ist, ist der Zeitpunkt, zu welchem die einzelnen Agenten anfangen den Spieler zu verfolgen bzw. die Verfolgung wieder aufgeben.

Ein Standardansatz ist es, allen Agenten einen Radius zu geben. Befindet sich der Spieler in diesem Radius wird er von dem Agenten detektiert und dieser nimmt die Verfolgung auf. Es reicht aber nicht, nur einen einzelnen Agenten loslaufen zu lassen. Damit das Verhalten schlüssig scheint, werden noch alle Agenten in einem gewissen Radius um den Agenten, der ursprünglich den Spieler detektiert hat, alarmiert.

Dieses Verhalten kann dann je nach gewünschten Verhalten iterativ für diese Agenten wiederholt werden. Damit wird simuliert, dass die Agenten untereinander kommunizieren, das Verhalten der Agenten wirkt natürlich und stellt den Spieler vor eine Herausforderung, da er nicht jeden Agenten einzeln besiegen kann, sondern sich direkt mit einer ganzen Gruppe konfrontiert sieht.

Stellt sich die Frage, ob der Radius, mit dem sich die Agenten gegenseitig alarmieren, der gleiche sein sollte, wie derjenige, mit dem der Agent den Spieler entdeckt hat.

Damit das Verhalten realistisch wirkt sollten die Agenten sich untereinander über eine größere Distanz alarmieren. Schließlich macht jeder Agent, der die Verfolgung aufnimmt, zusätzlich Lärm. So kann dieser Radius mit der Zahl der alarmierten Agenten wachsen, um den Lärm einer großen Gruppe zu simulieren.

Die Entwickler von Days Gone wenden einen ähnlichen Ansatz an. In dem Spiel wächst der Alarmierungsradius über die Zeit an, um zu verhindern, dass ein Zombie in einer Ecke der Szene übrig bleibt und der Spieler diesen suchen muss, um die Mission abzuschließen [7].

Die Radiusvergrößerung kann auch dazu führen, dass Agenten aus einer neuen Richtung alarmiert werden und damit den Spieler überraschen und vor Herausforderungen stellen, die schnelles und inspiriertes Handeln fordern.

Wenn sich eine Gruppe von Agenten gegenseitig alarmiert, sollte diese auch nicht zu perfekt erfolgen, indem alle synchron und sofort reagieren. Wenn die Agenten mit einer kleinen, am besten zufällig gewählten, Verzögerung reagieren, wirkt ihr Verhalten natürlicher. Dass alle Agenten die gleichen Animationen nutzen, fällt weniger auf, wenn sie nicht im gleichen Tick ihre Animationen beginnen (siehe Kapitel 7.3 „Animation und Modelle“).

7.3 ANIMATION UND MODELLE

Auch wenn er im Rahmen dieser Arbeit keine große Aufmerksamkeit erhalten hat, lohnt es sich dennoch, diesen Aspekt etwas zu beleuchten.

Animationen und Modelle der Agenten haben einen großen Einfluss auf die Performance, vor allem wenn sie hundert bis tausendfach gleichzeitig berechnet werden müssen. Gleichzeitig sollen auch nicht alle Agenten exakt gleich aussehen und die exakt gleichen Bewegungen im gleichen Rhythmus ausführen. Das würde die Immersion der Spieler erheblich stören.

Für die Modelle der Zombies habe die Entwickler von „Days Gone“ einen cleveren Ansatz gefunden, der nicht nur die Vielfalt der Zombies erhöht, ohne die Anzahl der Modelle zu erhöhen und außerdem dafür sorgt, dass die gleichen Animationen anders aussehen.

In „Days Gone“ gibt es 8 Modelle für die „Freaker“. Jedem dieser Freaker wird, wenn er geladen wird, eine zufällige Skalierung innerhalb des Rahmens [0.9 , 1.1] zugewiesen. Selbst wenn man diese Variationen in nur 3 Kategorien einteilt (hier: klein, mittel, groß) ergeben sich daraus 864 Kombinationsmöglichkeiten [7]. Passend zur Skalierung wird die Geschwindigkeit angepasst, wodurch auch die Lauf-Animation verändert wird.

Mit diesem simplen Trick brechen die Entwickler die uniforme Erscheinung einer großen Gruppe auf, ohne die Anzahl der Modelle und Animationen, die gleichzeitig geladen werden müssen, zu erhöhen. Beachtet man noch das im vorherigen Kapitel erläuterte Vorgehen, dass Agenten ihre Animationen leicht versetzt starten, kann sichergestellt werden, dass selbst wenn hunderte der gleichen Zombies mit der gleichen Animation den Spieler verfolgen, sich nie der Eindruck einstellt, von identischen Robotern verfolgt zu werden.

8 ZUSAMMENFASSUNG

Das Ziel dieser Arbeit war es möglichst ganzheitlich einen Ansatz der Navigation für eine Große Agentengruppe zu beschreiben und zu implementieren. Dabei hat sich gezeigt, dass es verschiedene Ansätze zu diesem Ziel gibt. Die Art des Spiels, die Szene, die Agenten und die Software sind entscheidende Faktoren dafür, welche Algorithmen das gewünschte Ergebnis produzieren. Besonders die Unterteilung der Szene in Knoten, die dann den Graphen für die Navigation bilden, bietet verschiedene Ansätze. Je nach dem ob nach Speicherplatz oder Rechenlast optimiert werden soll.

Es hat sich auch gezeigt, dass manche Teile der Lösung eine anerkannten Optimale Lösung haben. Die Bewegungsrichtung für die Agenten in Feldern zu Speichern, auf die diese dann nach Bedarf zugreifen, ist ab einer gewissen Anzahl an Angenten fast alternativlos.

Es hat sich auch gezeigt, dass mache Features, die moderne Game-Engines bereitstellen, den Entwicklern solcher Spiele sehr zugute kommen. Die Möglichkeit Nav-Meshes berechnen zu können ist eine elementare Grundlage, selbst für alternative Ansätze der Navigation.

Was die Leistungsfähigkeit der Algorithmen angeht, hat sich gezeigt, dass es durchaus möglich ist, eine vierstellige Zahl an Agenten zu bewältigen. Die Möglichkeiten weiter zu optimieren sind hier an allen Ecken und Enden zu finden. Manche dieser Möglichkeiten haben spezifisch mit der Anwendung und ihren Feinheiten zu tun, während andere sich um ganz elementare Themen wie effiziente Datenstrukturen und Berechnungen drehen.

Zuletzt lässt sich sagen, dass große Gruppen an Computergesteuerten Gegner ein hohes Potential für verschiedene Genres an Videospielen tragen. Es bleibt nur mit Spannung zu beobachten, welche Innovativen Ideen abseits der bekannten Archetypen die Zukunft der Videospieleentwicklung bereithält.

9 ABBILDUNGSVERZEICHNIS

Abbildung 1: Die Quadratischen Zellen (eigene Abbildung) 12

Abbildung 2: Dreiecke des Nav-Mesh mit unterschiedlichen Größen (eigenen Abbildung) 13

Abbildung 3: Beide Seiten der Wand liegen in der gleichen Zelle (eigenen Abbildung) 15

Abbildung 4: Das fertige Netz der Zellen (Eigene Abbildung) 18

Abbildung 5 Ein Vektorfeld, dass zu allen Agenten einen beinahe optimalen Pfad berechnet hat (Eigenen Abbildung) 23

Abbildung 6: Das Ergebnis von A*, wenn der Spieler (Rotes Quadrat) als Startpunkt gesetzt wird. Alle Erreichten Zellen enthalten einen Vektor der zum Spieler führt (Eigene Abbildung) 24

Abbildung 7: Das Ergebnis für eine Heuristik mit nur einem Ziel (Eigene Abbildung) 25

Abbildung 8: Das Ergebnis einer Heuristik mit zwei Zielen. (Eigene Abbildung) 25

Abbildung 9: Eine Zelle mit 6 Nachbarn (Links) Eine Zelle mit 26 Nachbarn (Rechts) (Eigenen Abbildung) 26

Abbildung 10 Das Wachstum bei steigender Anzahl der Zellen (Eigenen Abbildung) 29

Abbildung 11 Die Anzahl der Zellen mit zunehmender Größe der Zellen (Eigenen Abbildung) 29

Abbildung 12 Wachstum mit steigender Anzahl der Agenten (Eigenen Abbildung) 31

Abbildung 13 Steigende Zahl der Zellen bei steigender Zahl der Agenten (Eigenen Abbildung) 31

Abbildung 14 Die Ausführungszeit steigt mit der Summer der Zellen (Eigenen Abbildung) 33

Abbildung 15: DAS ERGEBNIS BEIM ERSTEN DURCHLAUF (EIGENE ABBILDUNG) 35

Abbildung 16: Ergebnis des zweiten Durchlaufs. Zu sehen sind nur jene Zellen, für die ein neuer Vektor berechnet wurde. (eigene Abbildung) 36

Abbildung 17: Der erste Durchgang (Links) der zweite Durchgang (Rechts) Das Portal ist in Rot zu sehen. (Eigene Abbildung) 37

Abbildung 18: Der Spieler hat sich ungefähr 10 Zellen zwischen den beiden Ausführungen von A* bewegt (Eigene Abbildung) 41

Abbildung 19 Die Agenten reihen sich auf dem Weg zum Ziel ein. (Eigene Abbildung) 43

Abbildung 20: Der Blockierte Bereich ist zu sehen (Eigene Abbildung) 45

Abbildung 21 Anwendung auf einer offenen Fläche. Spieler in Schwarz dargestellt. (Eigenen Abbildung) 46

Abbildung 22: Die Blockade fällt auf eine Engstelle. Der Spieler ist in Schwarz zu sehen. (Eigene Abbildung) 47

10 LITERATURVERZEICHNIS

[1] E. Emerson. "Crowd Pathfinding and Steering Using Flow Field Tiles." (n.d.). Accessed: Oct. 18, 2026. [Online]. Available: https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter23_Crowd_Pathfinding_and_Steering_Using_Flow_Field_Tiles.pdf

[2] S. Rabin and Sturtevant Nathan R., Eds. *Pathfinding Architecture*, 2013. Accessed: Oct. 24, 2026. [Online]. Available: https://www.gameaipro.com/GameAIPro/GameAIPro_Chapter17_Pathfinding_Architecture_Optimizations.pdf

[3] W. Ruml and J. T. Thayer, "Faster than Weighted A*: An Optimistic Approach to Bounded Suboptimal Search," 2008. [Online]. Available: https://cdn.aaai.org/ICAPS/2008/ICAPS08-044.pdf

[4] T. Akenine-Möller, "Fast 3D Triangle-Box Overlap Testing," 2001. [Online]. Available: https://fileadmin.cs.lth.se/cs/Personal/Tomas_Akenine-Moller/code/tribox_tam.pdf

[5] W. Sun, P. Tsiotras, and A. J. Yezzi, "Multiplayer Pursuit-Evasion Games in Three-Dimensional Flow Fields," *Dyn Games Appl*, vol. 9, no. 4, pp. 1188–1207, 2019, doi: 10.1007/s13235-019-00304-4.

[6] Unity. "Navigation Areas and Costs." Accessed: Oct. 23, 2026. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.ai.navigation@1.1/manual/AreasAndCosts.html

[7] Jon Ryan. "Days Gone: How Sony Bend Built the Horde." Accessed: 23.10. [Online]. Available: https://www.ign.com/articles/2019/04/08/days-gone-horde-making-of-bts

Alle Dokumente, Assets, Code etc. dieser Arbeit finden Sie unter:
https://github.com/funsimator/BA

11 ERKLÄRUNGEN

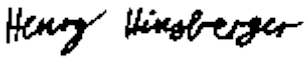
Die Unterzeichnung der Selbstständigkeitserklärung ist obligatorisch. Die Unterzeichnung der Ermächtigung ist optional.

11.1 SELBSTSTÄNDIGKEITSERKLÄRUNG

Hiermit versichere ich, dass ich die vorgelegte Arbeit selbstständig und ohne die Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Ich übernehme als Autorin oder Autor beim Einsatz von IT-/KI-gestützten Schreibwerkzeugen die Verantwortung für die inhaltliche Richtigkeit.

Hiermit erkläre ich, dass ich die aus fremden Quellen direkt oder indirekt übernommenen Gedanken als solche kenntlich gemacht habe.

Die Arbeit habe ich bisher keinem anderen Prüfungsamt in gleicher oder vergleichbarer Form vorgelegt. Sie wurde bisher nicht veröffentlicht.

Kirchseeon den 25.10.2026 

Ort, Datum Unterschrift

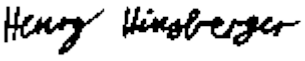
11.2 ERMÄCHTIGUNG

Die Urheberin/Der Urheber der studentischen Arbeit kann (muss nicht) erklären, dass die Hochschule Kempten folgende Nutzungsrechte erhält.

☒ Hiermit ermächtige ich/wir die Hochschule Kempten zur Veröffentlichung einer

Kurzzusammenfassung sowie Bilder/Screenshots und ggf. angefertigte Videos meiner studentischen Arbeit z. B. auf gedruckten Medien oder auf einer Internetseite der Hochschule Kempten zwecks Bewerbung des Bachelorstudiengangs „Game Engineering“ und des Masterstudiengangs „Game Engineering und Visual Computing“.

Dies betrifft insbesondere den Webauftritt der Hochschule Kempten und von der Hochschule betriebene Videokanäle. Die Hochschule Kempten erhält das einfache, unentgeltliche Nutzungsrecht im Sinne der §§ 31 Abs. 2, 32 Abs. 3 Satz 3 Urheberrechtsgesetz (UrhG).

Kirchseeon den 25.10.2026 

Ort, Datum Unterschrift